

Physlib Monthly Report

1–30 November 2024

Contributors: David Feng, Joseph Tooby-Smith

Reviewers: Joseph Tooby-Smith

Generated 14 August 2026 from commit [a57d38c](#).

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–30 November 2024 on branch `master` of [leanprover-community/physlib](#), between commits `16b3066` and `a57d38c`. Over that period 2 contributors landed 177 commits, changing 10,424 lines across 145 files and adding 469 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	3
2	Details by file	3
2.1	<code>HepLean.AnomalyCancellation.PureU1.Even.BasisLinear</code>	3
2.2	<code>HepLean.AnomalyCancellation.PureU1.Odd.BasisLinear</code>	6
2.3	<code>HepLean.FeynmanDiagrams.Wick.Algebra</code>	11
2.4	<code>HepLean.FeynmanDiagrams.Wick.Contract</code>	13
2.5	<code>HepLean.FeynmanDiagrams.Wick.Species</code>	20
2.6	<code>HepLean.FeynmanDiagrams.Wick.String</code>	20
2.7	<code>HepLean.Lorentz.Algebra.Basic</code>	23
2.8	<code>HepLean.Lorentz.ComplexTensor.Basis</code>	24
2.9	<code>HepLean.Lorentz.Group.Basic</code>	24
2.10	<code>HepLean.Lorentz.Group.Boosts</code>	25
2.11	<code>HepLean.Lorentz.MinkowskiMatrix</code>	25
2.12	<code>HepLean.Lorentz.RealVector.Basic</code>	27
2.13	<code>HepLean.Lorentz.RealVector.Contraction</code>	28
2.14	<code>HepLean.Lorentz.RealVector.Modules</code>	32
2.15	<code>HepLean.Lorentz.RealVector.NormOne</code>	38
2.16	<code>HepLean.Lorentz.SL2C.Basic</code>	42
2.17	<code>HepLean.Mathematics.Fin</code>	44
2.18	<code>HepLean.Meta.Basic</code>	45
2.19	<code>HepLean.Tensors.OverColor.Basic</code>	47
2.20	<code>HepLean.Tensors.OverColor.Discrete</code>	48

2.21	HepLean.Tensors.OverColor.Iso	49
2.22	HepLean.Tensors.OverColor.Lift	50
2.23	HepLean.Tensors.TensorSpecies.Basic	51
2.24	HepLean.Tensors.TensorSpecies.Contractions.Basic	52
2.25	HepLean.Tensors.TensorSpecies.Contractions.Categorical	53
2.26	HepLean.Tensors.TensorSpecies.DualRepIso	56
2.27	HepLean.Tensors.TensorSpecies.MetricTensor	58
2.28	HepLean.Tensors.TensorSpecies.Pure	59
2.29	HepLean.Tensors.TensorSpecies.UnitTensor	60
2.30	HepLean.Tensors.Tree.NodeIdentities.Assoc	62
2.31	HepLean.Tensors.Tree.NodeIdentities.Basic	62
2.32	HepLean.Tensors.Tree.NodeIdentities.ContrContr	62
2.33	HepLean.Tensors.Tree.NodeIdentities.ContrSwap	62
2.34	HepLean.Tensors.Tree.NodeIdentities.PermProd	63
2.35	HepLean.Tensors.Tree.NodeIdentities.ProdAssoc	63
2.36	HepLean.Tensors.Tree.NodeIdentities.ProdComm	63
2.37	HepLean.Tensors.Tree.NodeIdentities.ProdContr	64
2.38	scripts.MetaPrograms.no_docs	64
2.39	scripts.stats	64
2.40	scripts.Template	64

1 Introduction

During Nov 2024, 2 contributors submitted 177 commits that changed 10,424 lines (+7,231 / -3,193) across 145 files spanning 18 top-level areas of the repository. Of those, 23 files were newly added and 8 were removed outright. This report catalogues 469 newly added Lean declarations: 309 lemmas, 110 defs, 13 informal definitions, 13 informal lemmas, 11 instances, 5 inductives, 4 structures, and 4 abbrevs. We're delighted to recognize the following first-time contributor this month: [David Feng](#).

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
David Feng	8	2
Joseph Tooby-Smith	14,860	175

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

Reviewer	PRs Reviewed
Joseph Tooby-Smith	1

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
HepLean/Tensors	23	+9	—	+2,320	-688
HepLean/Lorentz	41	+7	—	+2,473	-348
HepLean/SpaceTime	10	—	-8	+15	-1,433
HepLean/FeynmanDiagrams	8	+4	—	+1,074	-12
HepLean/AnomalyCancellation	33	—	—	+569	-496
scripts	3	+1	—	+183	-66
HepLean/Meta	2	+1	—	+206	-2
HepLean/FlavorPhysics	6	—	—	+85	-25
HepLean.lean	1	—	—	+55	-42
HepLean/StandardModel	5	—	—	+60	-24
HepLean/Mathematics	4	—	—	+63	-9
README.md	1	—	—	+39	-19
scripts/MetaPrograms	2	+1	—	+45	-0
.github/workflows	2	—	—	+19	-12
lake-manifest.json	1	—	—	+13	-13
lakefile.toml	1	—	—	+7	-2
HepLean/BeyondTheStandardModel	1	—	—	+4	-1
lean-toolchain	1	—	—	+1	-1

1.4 New declarations by kind

Kind	Count
lemma	309
def	110
informal_definition	13
informal_lemma	13
instance	11
inductive	5
structure	4
abbrev	4

2 Details by file

2.1 HepLean.AnomalyCancellation.PureU1.Even.BasisLinear

[HepLean/AnomalyCancellation/PureU1/Even/BasisLinear.lean](#) on GitHub.

def evenFst

[\[source\]](#)

*The inclusion of $\text{Fin } n.\text{succ}$ into $\text{Fin } (n.\text{succ} + n.\text{succ})$ via the first $n.\text{succ}$, casted into $\text{Fin } (2 * n.\text{succ})$.*

```
def evenFst (j : Fin n.succ) : Fin (2 * n.succ)
```

def evenSnd

[\[source\]](#)

*The inclusion of $\text{Fin } n.\text{succ}$ into $\text{Fin } (n.\text{succ} + n.\text{succ})$ via the second $n.\text{succ}$, casted into $\text{Fin } (2 * n.\text{succ})$.*

```
def evenSnd (j : Fin n.succ) : Fin (2 * n.succ)
```

def evenShiftFst

[\[source\]](#)

*The inclusion of $\text{Fin } n$ into $\text{Fin } (1 + (n + n + 1))$ via the first n , casted into $\text{Fin } (2 * n.\text{succ})$.*

```
def evenShiftFst (j : Fin n) : Fin (2 * n.succ)
```

def evenShiftSnd

[\[source\]](#)

*The inclusion of $\text{Fin } n$ into $\text{Fin } (1 + (n + n + 1))$ via the second n , casted into $\text{Fin } (2 * n.\text{succ})$.*

```
def evenShiftSnd (j : Fin n) : Fin (2 * n.succ)
```

def evenShiftZero

[\[source\]](#)

*The element of $\text{Fin } (1 + (n + n + 1))$ corresponding to the first 1, casted into $\text{Fin } (2 * n.\text{succ})$.*

```
def evenShiftZero : Fin (2 * n.succ)
```

def evenShiftLast[\[source\]](#)

*The element of $\text{Fin } (1 + (n + n + 1))$ corresponding to the second 1, casted into $\text{Fin } (2 * n.\text{succ})$.*

```
def evenShiftLast : Fin (2 * n.succ)
```

lemma ext_even[\[source\]](#)

```
lemma ext_even (S T : Fin (2 * n.succ) → ℚ) (h1 : ∀ i, S (evenFst i) = T (evenFst i))
  (h2 : ∀ i, S (evenSnd i) = T (evenSnd i)) : S = T
```

lemma sum_even[\[source\]](#)

```
lemma sum_even (S : Fin (2 * n.succ) → ℚ) :
  ∑ i, S i = ∑ i : Fin n.succ, ((S ◦ evenFst) i + (S ◦ evenSnd) i)
```

lemma sum_evenShift[\[source\]](#)

```
lemma sum_evenShift (S : Fin (2 * n.succ) → ℚ) :
  ∑ i, S i = S evenShiftZero + S evenShiftLast +
  ∑ i : Fin n, ((S ◦ evenShiftFst) i + (S ◦ evenShiftSnd) i)
```

lemma evenShiftZero_eq_evenFst_zero[\[source\]](#)

```
lemma evenShiftZero_eq_evenFst_zero : @evenShiftZero n = evenFst 0
```

lemma evenShiftLast_eq_evenSnd_last[\[source\]](#)

```
lemma evenShiftLast_eq_evenSnd_last : @evenShiftLast n = evenSnd (Fin.last n)
```

lemma evenShiftFst_eq_evenFst_succ[\[source\]](#)

```
lemma evenShiftFst_eq_evenFst_succ (j : Fin n) : evenShiftFst j = evenFst j.succ
```

lemma evenShiftSnd_eq_evenSnd_castSucc[\[source\]](#)

```
lemma evenShiftSnd_eq_evenSnd_castSucc (j : Fin n) : evenShiftSnd j = evenSnd j.castSucc
```

lemma basis_on_evenFst_self[\[source\]](#)

```
lemma basis_on_evenFst_self (j : Fin n.succ) : basisAsCharges j (evenFst j) = 1
```

lemma basis!_on_evenShiftFst_self[\[source\]](#)

```
lemma basis!_on_evenShiftFst_self (j : Fin n) : basis!AsCharges j (evenShiftFst j) = 1
```

lemma basis_on_evenFst_other[\[source\]](#)

```
lemma basis_on_evenFst_other {k j : Fin n.succ} (h : k ≠ j) :
  basisAsCharges k (evenFst j) = 0
```

lemma basis!_on_evenShiftFst_other[\[source\]](#)

```
lemma basis!_on_evenShiftFst_other {k j : Fin n} (h : k ≠ j) :
  basis!AsCharges k (evenShiftFst j) = 0
```

lemma basis_evenSnd_eq_neg_evenFst[\[source\]](#)

```
lemma basis_evenSnd_eq_neg_evenFst (j i : Fin n.succ) :
  basisAsCharges j (evenSnd i) = - basisAsCharges j (evenFst i)
```

lemma basis!_evenShftSnd_eq_neg_evenShiftFst[\[source\]](#)

```
lemma basis!_evenShftSnd_eq_neg_evenShiftFst (j i : Fin n) :
  basis!AsCharges j (evenShiftSnd i) = - basis!AsCharges j (evenShiftFst i)
```

lemma basis_on_evenSnd_self[\[source\]](#)

```
lemma basis_on_evenSnd_self (j : Fin n.succ) : basisAsCharges j (evenSnd j) = - 1
```

lemma basis!_on_evenShiftSnd_self[\[source\]](#)

```
lemma basis!_on_evenShiftSnd_self (j : Fin n) : basis!AsCharges j (evenShiftSnd j) = - 1
```

lemma basis_on_evenSnd_other[\[source\]](#)

```
lemma basis_on_evenSnd_other {k j : Fin n.succ} (h : k ≠ j) : basisAsCharges k (evenSnd j) = 0
```

lemma basis!_on_evenShiftSnd_other[\[source\]](#)

```
lemma basis!_on_evenShiftSnd_other {k j : Fin n} (h : k ≠ j) :
  basis!AsCharges k (evenShiftSnd j) = 0
```

lemma basis!_on_evenShiftZero[\[source\]](#)

```
lemma basis!_on_evenShiftZero (j : Fin n) : basis!AsCharges j evenShiftZero = 0
```

lemma basis!_on_evenShiftLast[\[source\]](#)

```
lemma basis!_on_evenShiftLast (j : Fin n) : basis!AsCharges j evenShiftLast = 0
```

lemma P_evenFst[\[source\]](#)

```
lemma P_evenFst (f : Fin n.succ → ℚ) (j : Fin n.succ) : P f (evenFst j) = f j
```

lemma P!_evenShiftFst[\[source\]](#)

```
lemma P!_evenShiftFst (f : Fin n → ℚ) (j : Fin n) : P! f (evenShiftFst j) = f j
```

lemma Pa_evenShiftFst[\[source\]](#)

```
lemma Pa_evenShiftFst (f : Fin n.succ → ℚ) (g : Fin n → ℚ) (j : Fin n) :  
  Pa f g (evenShiftFst j) = f j.succ + g j
```

lemma P_evenSnd[\[source\]](#)

```
lemma P_evenSnd (f : Fin n.succ → ℚ) (j : Fin n.succ) : P f (evenSnd j) = - f j
```

lemma P!_evenShiftSnd[\[source\]](#)

```
lemma P!_evenShiftSnd (f : Fin n → ℚ) (j : Fin n) : P! f (evenShiftSnd j) = - f j
```

lemma Pa_evenShiftSnd[\[source\]](#)

```
lemma Pa_evenShiftSnd (f : Fin n.succ → ℚ) (g : Fin n → ℚ) (j : Fin n) :  
  Pa f g (evenShiftSnd j) = - f j.castSucc - g j
```

lemma P!_evenShiftZero[\[source\]](#)

```
lemma P!_evenShiftZero (f : Fin n → ℚ) : P! f (evenShiftZero) = 0
```

lemma Pa_evenShitZero[\[source\]](#)

```
lemma Pa_evenShitZero (f : Fin n.succ → ℚ) (g : Fin n → ℚ) : Pa f g (evenShiftZero) = f 0
```

lemma P!_evenShiftLast[\[source\]](#)

```
lemma P!_evenShiftLast (f : Fin n → ℚ) : P! f (evenShiftLast) = 0
```

lemma Pa_evenShiftLast[\[source\]](#)

```
lemma Pa_evenShiftLast (f : Fin n.succ → ℚ) (g : Fin n → ℚ) :  
  Pa f g (evenShiftLast) = - f (Fin.last n)
```

lemma P_evenSnd_evenFst[\[source\]](#)

```
lemma P_evenSnd_evenFst (f : Fin n.succ → ℚ) : P f ∘ evenSnd = - P f ∘ evenFst
```

2.2 HepLean.AnomalyCancellation.PureU1.Odd.BasisLinear

[HepLean/AnomalyCancellation/PureU1/Odd/BasisLinear.lean](#) on GitHub.

lemma odd_shift_eq[\[source\]](#)

```
lemma odd_shift_eq (n : ℕ) : (1 + n) + n = 2 * n + 1
```

lemma odd_shift_shift_eq[\[source\]](#)

```
lemma odd_shift_shift_eq (n : ℕ) : ((1+n)+1) + n.succ = 2 * n.succ + 1
```

def oddFst[\[source\]](#)

*The inclusion of $\text{Fin } n$ into $\text{Fin } ((n + 1) + n)$ via the first n . This is then casted to $\text{Fin } (2 * n + 1)$.*

```
def oddFst (j : Fin n) : Fin (2 * n + 1)
```

def oddSnd[\[source\]](#)

*The inclusion of $\text{Fin } n$ into $\text{Fin } ((n + 1) + n)$ via the second n . This is then casted to $\text{Fin } (2 * n + 1)$.*

```
def oddSnd (j : Fin n) : Fin (2 * n + 1)
```

def oddMid[\[source\]](#)

*The element representing 1 in $\text{Fin } ((n + 1) + n)$. This is then casted to $\text{Fin } (2 * n + 1)$.*

```
def oddMid : Fin (2 * n + 1)
```

def oddShiftFst[\[source\]](#)

*The inclusion of $\text{Fin } n$ into $\text{Fin } (1 + n + n)$ via the first n . This is then casted to $\text{Fin } (2 * n + 1)$.*

```
def oddShiftFst (j : Fin n) : Fin (2 * n + 1)
```

def oddShiftSnd[\[source\]](#)

*The inclusion of $\text{Fin } n$ into $\text{Fin } (1 + n + n)$ via the second n . This is then casted to $\text{Fin } (2 * n + 1)$.*

```
def oddShiftSnd (j : Fin n) : Fin (2 * n + 1)
```

def oddShiftZero[\[source\]](#)

*The element representing the 1 in $\text{Fin } (1 + n + n)$. This is then casted to $\text{Fin } (2 * n + 1)$.*

```
def oddShiftZero : Fin (2 * n + 1)
```

def oddShiftShiftZero[\[source\]](#)

*The element representing the first 1 in $\text{Fin } (1 + n + 1 + n.succ)$ casted to $\text{Fin } (2 * n.succ + 1)$.*

```
def oddShiftShiftZero : Fin (2 * n.succ + 1)
```

def oddShiftShiftFst[\[source\]](#)

*The inclusion of $\text{Fin } n$ into $\text{Fin } (1 + n + 1 + n.\text{succ})$ via the first n and casted to $\text{Fin } (2 * n.\text{succ} + 1)$.*

```
def oddShiftShiftFst (j : Fin n) : Fin (2 * n.succ + 1)
```

def oddShiftShiftMid[\[source\]](#)

*The element representing the second 1 in $\text{Fin } (1 + n + 1 + n.\text{succ})$ casted to $2 * n.\text{succ} + 1$.*

```
def oddShiftShiftMid : Fin (2 * n.succ + 1)
```

def oddShiftShiftSnd[\[source\]](#)

*The inclusion of $\text{Fin } n.\text{succ}$ into $\text{Fin } (1 + n + 1 + n.\text{succ})$ via the $n.\text{succ}$ and casted to $\text{Fin } (2 * n.\text{succ} + 1)$.*

```
def oddShiftShiftSnd (j : Fin n.succ) : Fin (2 * n.succ + 1)
```

lemma oddShiftShiftZero_eq_oddFst_zero[\[source\]](#)

```
lemma oddShiftShiftZero_eq_oddFst_zero : @oddShiftShiftZero n = oddFst 0
```

lemma oddShiftShiftZero_eq_oddShiftZero[\[source\]](#)

```
lemma oddShiftShiftZero_eq_oddShiftZero : @oddShiftShiftZero n = oddShiftZero
```

lemma oddShiftShiftFst_eq_oddFst_succ[\[source\]](#)

```
lemma oddShiftShiftFst_eq_oddFst_succ (j : Fin n) :  
  oddShiftShiftFst j = oddFst j.succ
```

lemma oddShiftShiftFst_eq_oddShiftFst_castSucc[\[source\]](#)

```
lemma oddShiftShiftFst_eq_oddShiftFst_castSucc (j : Fin n) :  
  oddShiftShiftFst j = oddShiftFst j.castSucc
```

lemma oddShiftShiftMid_eq_oddMid[\[source\]](#)

```
lemma oddShiftShiftMid_eq_oddMid : @oddShiftShiftMid n = oddMid
```

lemma oddShiftShiftMid_eq_oddShiftFst_last[\[source\]](#)

```
lemma oddShiftShiftMid_eq_oddShiftFst_last : oddShiftShiftMid = oddShiftFst (Fin.last n)
```

lemma oddShiftShiftSnd_eq_oddSnd[\[source\]](#)

```
lemma oddShiftShiftSnd_eq_oddSnd (j : Fin n.succ) : oddShiftShiftSnd j = oddSnd j
```

lemma oddShiftShiftSnd_eq_oddShiftSnd[\[source\]](#)

```
lemma oddShiftShiftSnd_eq_oddShiftSnd (j : Fin n.succ) : oddShiftShiftSnd j = oddShiftSnd j
```

lemma oddSnd_eq_oddShiftSnd[\[source\]](#)

```
lemma oddSnd_eq_oddShiftSnd (j : Fin n) : oddSnd j = oddShiftSnd j
```

lemma sum_odd[\[source\]](#)

```
lemma sum_odd (S : Fin (2 * n + 1) → ℚ) :
  ∑ i, S i = S oddMid + ∑ i : Fin n, ((S ◦ oddFst) i + (S ◦ oddSnd) i)
```

lemma sum_oddShift[\[source\]](#)

```
lemma sum_oddShift (S : Fin (2 * n + 1) → ℚ) :
  ∑ i, S i = S oddShiftZero + ∑ i : Fin n, ((S ◦ oddShiftFst) i + (S ◦ oddShiftSnd) i)
```

lemma basis_on_oddFst_self[\[source\]](#)

```
lemma basis_on_oddFst_self (j : Fin n) : basisAsCharges j (oddFst j) = 1
```

lemma basis!_on_oddShiftFst_self[\[source\]](#)

```
lemma basis!_on_oddShiftFst_self (j : Fin n) : basis!AsCharges j (oddShiftFst j) = 1
```

lemma basis_on_oddFst_other[\[source\]](#)

```
lemma basis_on_oddFst_other {k j : Fin n} (h : k ≠ j) :
  basisAsCharges k (oddFst j) = 0
```

lemma basis!_on_oddShiftFst_other[\[source\]](#)

```
lemma basis!_on_oddShiftFst_other {k j : Fin n} (h : k ≠ j) :
  basis!AsCharges k (oddShiftFst j) = 0
```

lemma basis_oddSnd_eq_minus_oddFst[\[source\]](#)

```
lemma basis_oddSnd_eq_minus_oddFst (j i : Fin n) :
  basisAsCharges j (oddSnd i) = - basisAsCharges j (oddFst i)
```

lemma basis!_oddShiftSnd_eq_minus_oddShiftFst[\[source\]](#)

```
lemma basis!_oddShiftSnd_eq_minus_oddShiftFst (j i : Fin n) :
  basis!AsCharges j (oddShiftSnd i) = - basis!AsCharges j (oddShiftFst i)
```

lemma basis_on_oddSnd_self[\[source\]](#)

```
lemma basis_on_oddSnd_self (j : Fin n) : basisAsCharges j (oddSnd j) = - 1
```

lemma basis!_on_oddShiftSnd_self[\[source\]](#)

```
lemma basis!_on_oddShiftSnd_self (j : Fin n) : basis!AsCharges j (oddShiftSnd j) = - 1
```

lemma basis_on_oddSnd_other[\[source\]](#)

```
lemma basis_on_oddSnd_other {k j : Fin n} (h : k ≠ j) : basisAsCharges k (oddSnd j) = 0
```

lemma basis!_on_oddShiftSnd_other[\[source\]](#)

```
lemma basis!_on_oddShiftSnd_other {k j : Fin n} (h : k ≠ j) :
  basis!AsCharges k (oddShiftSnd j) = 0
```

lemma basis_on_oddMid[\[source\]](#)

```
lemma basis_on_oddMid (j : Fin n) : basisAsCharges j oddMid = 0
```

lemma basis!_on_oddShiftZero[\[source\]](#)

```
lemma basis!_on_oddShiftZero (j : Fin n) : basis!AsCharges j oddShiftZero = 0
```

lemma P_oddFst[\[source\]](#)

```
lemma P_oddFst (f : Fin n → ℚ) (j : Fin n) : P f (oddFst j) = f j
```

lemma P!_oddShiftFst[\[source\]](#)

```
lemma P!_oddShiftFst (f : Fin n → ℚ) (j : Fin n) : P! f (oddShiftFst j) = f j
```

lemma P_oddSnd[\[source\]](#)

```
lemma P_oddSnd (f : Fin n → ℚ) (j : Fin n) : P f (oddSnd j) = - f j
```

lemma P!_oddShiftSnd[\[source\]](#)

```
lemma P!_oddShiftSnd (f : Fin n → ℚ) (j : Fin n) : P! f (oddShiftSnd j) = - f j
```

lemma P_oddMid[\[source\]](#)

```
lemma P_oddMid (f : Fin n → ℚ) : P f (oddMid) = 0
```

lemma P!_oddShiftZero[\[source\]](#)

```
lemma P!_oddShiftZero (f : Fin n → ℚ) : P! f (oddShiftZero) = 0
```

lemma Pa_oddShiftShiftZero[\[source\]](#)

```
lemma Pa_oddShiftShiftZero (f g : Fin n.succ → ℚ) : Pa f g oddShiftShiftZero = f 0
```

lemma Pa_oddShiftShiftFst[\[source\]](#)

```
lemma Pa_oddShiftShiftFst (f g : Fin n.succ → ℚ) (j : Fin n) :
  Pa f g (oddShiftShiftFst j) = f j.succ + g j.castSucc
```

lemma Pa_oddShiftShiftMid[\[source\]](#)

```
lemma Pa_oddShiftShiftMid (f g : Fin n.succ → ℚ) : Pa f g (oddShiftShiftMid) = g (Fin.last n)
```

lemma Pa_oddShiftShiftSnd[\[source\]](#)

```
lemma Pa_oddShiftShiftSnd (f g : Fin n.succ → ℚ) (j : Fin n.succ) :
  Pa f g (oddShiftShiftSnd j) = - f j - g j
```

2.3 HepLean.FeynmanDiagrams.Wick.Algebra

[HepLean/FeynmanDiagrams/Wick/Algebra.lean](#) on GitHub.

informal_definition WickAlgebra[\[source\]](#)

```
informal_definition WickAlgebra where
  math := "
    Modifications of this may be needed.
    A structure with the following data:
    - A  $\mathbb{Z}_2$ -graded algebra  $A$ .
    - A map from  $\psi : \mathcal{E} \times \text{SpaceTime} \rightarrow A$  where  $\mathcal{E}$  are field colors.
    - A map  $\psi_+ : \mathcal{E} \times \text{SpaceTime} \rightarrow A$ .
    - A map  $\psi_- : \mathcal{E} \times \text{SpaceTime} \rightarrow A$ .
    Subject to the conditions:
    - The sum of  $\psi_0$  and  $\psi_1$  is  $\psi$ .
    - Two fields super-commute if there colors are not dual to each other.
    - The super-commutator of two fields is always in the
      center of the algebra. "
  physics := "This is defined to be an
    abstraction of the notion of an operator algebra."
  ref := "https://physics.stackexchange.com/questions/24157/"
```

informal_definition WickMonomial[\[source\]](#)

```
informal_definition WickMonomial where
  math := "The type of elements of the Wick algebra which is a product of fields."
  deps := [``WickAlgebra]
```

informal_definition toWickAlgebra[\[source\]](#)

```
informal_definition toWickAlgebra where
  math := "A function from WickMonomial to WickAlgebra which takes a monomial and
    returns the product of the fields in the monomial."
  deps := [``WickAlgebra, ``WickMonomial]
```

informal_definition timeOrder[\[source\]](#)

```
informal_definition timeOrder where
  math := "A function from WickMonomial to WickAlgebra which takes a monomial and
```

```

    returns the monomial with the fields time ordered, with the correct sign
    determined by the Koszul sign factor."
deps := [``WickAlgebra, ``WickMonomial]

```

informal_definition normalOrder[\[source\]](#)

```

informal_definition normalOrder where
  math := "A function from WickMonomial to WickAlgebra which takes a monomial and
    returns the element in `WickAlgebra` defined as follows
    - The  $\psi_+$  fields are move to the right.
    - The  $\psi_-$  fields are moved to the left.
    - Otherwise the order of the fields is preserved."
  ref := "https://www.imperial.ac.uk/media/imperial-college/research-centres-and-groups/
    theoretical-physics/msc/current/qft/handouts/qftwickstheorem.pdf"
  deps := [``WickAlgebra, ``WickMonomial]

```

informal_definition contraction[\[source\]](#)

```

informal_definition contraction where
  math := "Given two  $i\ j : \mathcal{E} \times \text{SpaceTime}$ , the element of WickAlgebra
    defined by subtracting the normal ordering of  $\psi\ i\ \psi\ j$  from the time-ordering of
     $\psi\ i\ \psi\ j$ ."
  deps := [``WickAlgebra, ``WickMonomial]

```

informal_lemma contraction_in_center[\[source\]](#)

```

informal_lemma contraction_in_center where
  math := "The contraction of two fields is in the center of the algebra."
  deps := [``WickAlgebra, ``contraction]

```

informal_lemma contraction_non_dual_is_zero[\[source\]](#)

```

informal_lemma contraction_non_dual_is_zero where
  math := "The contraction of two fields is zero if the fields are not dual to each other."
  deps := [``WickAlgebra, ``contraction]

```

informal_lemma timeOrder_single[\[source\]](#)

```

informal_lemma timeOrder_single where
  math := "The time ordering of a single field is the normal ordering of that field."
  proof := "Follows from the definitions."
  deps := [``WickAlgebra, ``WickMonomial.timeOrder, ``WickMonomial.normalOrder]

```

informal_lemma timeOrder_pair[\[source\]](#)

```

informal_lemma timeOrder_pair where
  math := "The time ordering of two fields is the normal ordering of the fields plus the
    contraction of the fields."
  proof := "Follows from the definition of contraction."
  deps := [``WickAlgebra, ``WickMonomial.timeOrder, ``WickMonomial.normalOrder,
    ``contraction]

```

2.4 HepLean.FeynmanDiagrams.Wick.Contract

[HepLean/FeynmanDiagrams/Wick/Contract.lean](#) on GitHub.

inductive WickContract

[\[source\]](#)

A Wick contraction for a Wick string is a series of pairs i and j of indices to be contracted, subject to ordering and subject to the condition that they can be contracted.

```
inductive WickContract : {ni : ℕ} → {i : Fin ni → ℰ} → {n : ℕ} → {c : Fin n → ℰ} →
  {no : ℕ} → {o : Fin no → ℰ} →
  (str : WickString i c o final) →
  {k : ℕ} → (b1 : Fin k → Fin n) → (b2 : Fin k → Fin n) → Type where
| string {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final} : WickContract str Fin.elim0 Fin.
  elim0
| contr {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final} {k : ℕ}
  {b1 : Fin k → Fin n} {b2 : Fin k → Fin n} : (i : Fin n) →
  (j : Fin n) → (h : c j = ξ (c i)) →
  (hilej : i < j) → (hb1 : ∀ r, b1 r < i) → (hb2i : ∀ r, b2 r ≠ i) → (hb2j : ∀ r, b2 r ≠ j) →
  (w : WickContract str b1 b2) →
  WickContract str (Fin.snoc b1 i) (Fin.snoc b2 j)
```

def size

[\[source\]](#)

The number of nodes of a Wick contraction.

```
def size {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final} {k : ℕ} {b1 b2 : Fin k → Fin n} :
  WickContract str b1 b2 → ℕ
```

lemma size_eq_k

[\[source\]](#)

The number of nodes in a wick contraction tree is the same as k .

```
lemma size_eq_k {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final} {k : ℕ} {b1 b2 : Fin k → Fin n} :
  (w : WickContract str b1 b2) → w.size = k
```

def boundFst

[\[source\]](#)

The map giving the vertices on the left-hand-side of a contraction.

```
@[nolint unusedArguments]
def boundFst {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} :
  WickContract str b1 b2 → Fin k → Fin n
```

lemma boundFst_constr_castSucc

[\[source\]](#)

```
@[simp]
lemma boundFst_constr_castSucc {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} (i j : Fin n)
```

```

(h : c j =  $\xi$  (c i))
(hilej : i < j)
(hb1 :  $\forall$  r, b1 r < i)
(hb2i :  $\forall$  r, b2 r  $\neq$  i)
(hb2j :  $\forall$  r, b2 r  $\neq$  j)
(w : WickContract str b1 b2) (r : Fin k) :
(contr i j h hilej hb1 hb2i hb2j w).boundFst r.castSucc = w.boundFst r

```

lemma boundFst_contr_last[\[source\]](#)

```

@[simp]
lemma boundFst_contr_last {ni :  $\mathbb{N}$ } {i : Fin ni  $\rightarrow$   $\mathcal{E}$ } {n :  $\mathbb{N}$ } {c : Fin n  $\rightarrow$   $\mathcal{E}$ }
  {no :  $\mathbb{N}$ } {o : Fin no  $\rightarrow$   $\mathcal{E}$ } {str : WickString i c o final}
  {k :  $\mathbb{N}$ } {b1 b2 : Fin k  $\rightarrow$  Fin n} (i j : Fin n)
  (h : c j =  $\xi$  (c i))
  (hilej : i < j)
  (hb1 :  $\forall$  r, b1 r < i)
  (hb2i :  $\forall$  r, b2 r  $\neq$  i)
  (hb2j :  $\forall$  r, b2 r  $\neq$  j)
  (w : WickContract str b1 b2) :
  (contr i j h hilej hb1 hb2i hb2j w).boundFst (Fin.last k) = i

```

lemma boundFst_strictMono[\[source\]](#)

```

lemma boundFst_strictMono {ni :  $\mathbb{N}$ } {i : Fin ni  $\rightarrow$   $\mathcal{E}$ } {n :  $\mathbb{N}$ } {c : Fin n  $\rightarrow$   $\mathcal{E}$ }
  {no :  $\mathbb{N}$ } {o : Fin no  $\rightarrow$   $\mathcal{E}$ } {str : WickString i c o final}
  {k :  $\mathbb{N}$ } {b1 b2 : Fin k  $\rightarrow$  Fin n} : (w : WickContract str b1 b2)  $\rightarrow$  StrictMono w.boundFst

```

def boundSnd[\[source\]](#)

The map giving the vertices on the right-hand-side of a contraction.

```

@[nolint unusedArguments]
def boundSnd {ni :  $\mathbb{N}$ } {i : Fin ni  $\rightarrow$   $\mathcal{E}$ } {n :  $\mathbb{N}$ } {c : Fin n  $\rightarrow$   $\mathcal{E}$ }
  {no :  $\mathbb{N}$ } {o : Fin no  $\rightarrow$   $\mathcal{E}$ } {str : WickString i c o final}
  {k :  $\mathbb{N}$ } {b1 b2 : Fin k  $\rightarrow$  Fin n} :
  WickContract str b1 b2  $\rightarrow$  Fin k  $\rightarrow$  Fin n

```

lemma boundSnd_contr_castSucc[\[source\]](#)

```

@[simp]
lemma boundSnd_contr_castSucc {ni :  $\mathbb{N}$ } {i : Fin ni  $\rightarrow$   $\mathcal{E}$ } {n :  $\mathbb{N}$ } {c : Fin n  $\rightarrow$   $\mathcal{E}$ }
  {no :  $\mathbb{N}$ } {o : Fin no  $\rightarrow$   $\mathcal{E}$ } {str : WickString i c o final}
  {k :  $\mathbb{N}$ } {b1 b2 : Fin k  $\rightarrow$  Fin n} (i j : Fin n)
  (h : c j =  $\xi$  (c i))
  (hilej : i < j)
  (hb1 :  $\forall$  r, b1 r < i)
  (hb2i :  $\forall$  r, b2 r  $\neq$  i)
  (hb2j :  $\forall$  r, b2 r  $\neq$  j)
  (w : WickContract str b1 b2) (r : Fin k) :
  (contr i j h hilej hb1 hb2i hb2j w).boundSnd r.castSucc = w.boundSnd r

```

lemma boundSnd_contr_last[\[source\]](#)

@[simp]

```

lemma boundSnd_contr_last {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} (i j : Fin n)
  (h : c j = ξ (c i))
  (hilej : i < j)
  (hb1 : ∀ r, b1 r < i)
  (hb2i : ∀ r, b2 r ≠ i)
  (hb2j : ∀ r, b2 r ≠ j)
  (w : WickContract str b1 b2) :
  (contr i j h hilej hb1 hb2i hb2j w).boundSnd (Fin.last k) = j

```

lemma boundSnd_injective

[\[source\]](#)

```

lemma boundSnd_injective {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} :
  (w : WickContract str b1 b2) → Function.Injective w.boundSnd

```

lemma color_boundSnd_eq_dual_boundFst

[\[source\]](#)

```

lemma color_boundSnd_eq_dual_boundFst {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} :
  (w : WickContract str b1 b2) → (i : Fin k) → c (w.boundSnd i) = ξ (c (w.boundFst i))

```

lemma boundFst_lt_boundSnd

[\[source\]](#)

```

lemma boundFst_lt_boundSnd {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} : (w : WickContract str b1 b2) → (i : Fin k) →
  w.boundFst i < w.boundSnd i

```

lemma boundFst_neq_boundSnd

[\[source\]](#)

```

lemma boundFst_neq_boundSnd {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} :
  (w : WickContract str b1 b2) → (r1 r2 : Fin k) → b1 r1 ≠ b2 r2

```

def castMaps

[\[source\]](#)

Casts a Wick contraction from WickContract str b1 b2 to WickContract str b1' b2' with a proof that b1 = b1' and b2 = b2', and that they are defined from the same k = k'.

```

def castMaps {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k k' : ℕ} {b1 b2 : Fin k → Fin n} {b1' b2' : Fin k' → Fin n}
  (hk : k = k')
  (hb1 : b1 = b1' ∘ Fin.cast hk) (hb2 : b2 = b2' ∘ Fin.cast hk) (w : WickContract str b1 b2)
  :
  WickContract str b1' b2'

```

lemma castMaps_rfl[\[source\]](#)

```
@[simp]
lemma castMaps_rfl {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} (w : WickContract str b1 b2) :
  castMaps rfl rfl rfl w = w
```

lemma mem_snoc'[\[source\]](#)

```
lemma mem_snoc' {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1' b2' : Fin k → Fin n} :
  (w : WickContract str b1' b2') →
  {k' : ℕ} → (hk' : k'.succ = k) →
  (b1 b2 : Fin k' → Fin n) → (i j : Fin n) → (h : c j = ξ (c i)) →
  (hilej : i < j) → (hb1 : ∀ r, b1 r < i) → (hb2i : ∀ r, b2 r ≠ i) → (hb2j : ∀ r, b2 r ≠ j) →
  (hb1' : Fin.snoc b1 i = b1' ∘ Fin.cast hk') →
  (hb2' : Fin.snoc b2 j = b2' ∘ Fin.cast hk') →
  ∃ (w' : WickContract str b1 b2), w = castMaps hk' hb1' hb2' (
    contr i j h hilej hb1 hb2i hb2j w')
```

lemma mem_snoc[\[source\]](#)

```
lemma mem_snoc {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (i j : Fin n) (h : c j = ξ (c i)) (hilej : i < j) (hb1 : ∀ r, b1 r < i)
  (hb2i : ∀ r, b2 r ≠ i) (hb2j : ∀ r, b2 r ≠ j)
  (w : WickContract str (Fin.snoc b1 i) (Fin.snoc b2 j)) :
  ∃ (w' : WickContract str b1 b2), w = contr i j h hilej hb1 hb2i hb2j w'
```

lemma is_subsingleton[\[source\]](#)

```
lemma is_subsingleton {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} :
  Subsingleton (WickContract str b1 b2)
```

lemma eq_snoc_castSucc[\[source\]](#)

```
lemma eq_snoc_castSucc {k n : ℕ} (b1 : Fin k.succ → Fin n) :
  b1 = Fin.snoc (b1 ∘ Fin.castSucc) (b1 (Fin.last k))
```

def fromMaps[\[source\]](#)

The construction of a Wick contraction from maps $b1\ b2 : Fin\ k \rightarrow Fin\ n$, with the former giving the first index to be contracted, and the latter the second index. These maps must satisfy a series of conditions.

```
def fromMaps {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} (b1 b2 : Fin k → Fin n)
  (hi : ∀ i, c (b2 i) = ξ (c (b1 i)))
  (hb1ltb2 : ∀ i, b1 i < b2 i)
```

```
(hb1 : StrictMono b1)
(hb1neb2 : ∀ r1 r2, b1 r1 ≠ b2 r2)
(hb2 : Function.Injective b2) :
WickContract str b1 b2
```

def dropLast[\[source\]](#)

Given a Wick contraction with k .succ contractions, returns the Wick contraction with k contractions by dropping the last contraction (defined by the first index contracted).

```
def dropLast {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
{no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
{k : ℕ} {b1 b2 : Fin k.succ → Fin n}
(w : WickContract str b1 b2) : WickContract str (b1 ∘ Fin.castSucc) (b2 ∘ Fin.castSucc)
```

lemma eq_from_maps[\[source\]](#)

```
lemma eq_from_maps {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
{no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
{k : ℕ} {b1 b2 : Fin k → Fin n}
(w : WickContract str b1 b2) :
w = fromMaps w.boundFst w.boundSnd w.color_boundSnd_eq_dual_boundFst
  w.boundFst_lt_boundSnd w.boundFst_strictMono w.boundFst_neq_boundSnd
  w.boundSnd_injective
```

lemma eq_dropLast_contr[\[source\]](#)

```
lemma eq_dropLast_contr {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
{no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
{k : ℕ} {b1 b2 : Fin k.succ → Fin n} (w : WickContract str b1 b2) :
w = castMaps rfl (eq_snoc_castSucc b1).symm (eq_snoc_castSucc b2).symm
  (contr (b1 (Fin.last k)) (b2 (Fin.last k))
    (w.color_boundSnd_eq_dual_boundFst (Fin.last k))
    (w.boundFst_lt_boundSnd (Fin.last k))
    (fun r => w.boundFst_strictMono (Fin.castSucc_lt_last r))
    (fun r a => w.boundFst_neq_boundSnd (Fin.last k) r.castSucc a.symm)
    (fun r => w.boundSnd_injective.eq_iff.mp.mt (Fin.ne_last_of_lt (Fin.castSucc_lt_last r))))
  (dropLast w))
```

def Level[\[source\]](#)

Wick contractions of a given Wick string with k different contractions.

```
def Level {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
{no : ℕ} {o : Fin no → ℰ} (str : WickString i c o final) (k : ℕ) : Type
```

instance levelZeroFintype[\[source\]](#)

There is a finite number of Wick contractions with no contractions. In particular, this is just the original Wick string.

```
instance levelZeroFintype {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
{no : ℕ} {o : Fin no → ℰ} (str : WickString i c o final) :
Fintype (Level str 0) where
```

structure ContrPair[\[source\]](#)

The pairs of additional indices which can be contracted given a Wick contraction.

```
structure ContrPair {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) where
  i : Fin n
  j : Fin n
  h : c j = ξ (c i)
  hilej : i < j
  hb1 : ∀ r, b1 r < i
  hb2i : ∀ r, b2 r ≠ i
  hb2j : ∀ r, b2 r ≠ j
```

def contrPairEquivSubtype[\[source\]](#)

The pairs of additional indices which can be contracted, given an existing wick contraction, is equivalent to the a subtype of $\text{Fin } n \times \text{Fin } n$ defined by certain conditions equivalent to the conditions appearing in `ContrPair`.

```
def contrPairEquivSubtype {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} (w : WickContract str b1 b2) :
  ContrPair w ≈ {x : Fin n × Fin n // c x.2 = ξ (c x.1) ∧ x.1 < x.2 ∧
    (∀ r, b1 r < x.1) ∧ (∀ r, b2 r ≠ x.1) ∧ (∀ r, b2 r ≠ x.2)} where
```

lemma heq_eq[\[source\]](#)

```
lemma heq_eq {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 b1' b2' : Fin k → Fin n}
  (w : WickContract str b1 b2)
  (w' : WickContract str b1' b2') (h1 : b1 = b1') (h2 : b2 = b2') : HEq w w'
```

def levelSuccEquiv[\[source\]](#)

The equivalence between Wick contractions consisting of `k.succ` contractions and those with `k` contractions paired with a suitable contraction pair.

```
def levelSuccEquiv {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} (str : WickString i c o final) (k : ℕ) :
  Level str k.succ ≈ (w : Level str k) × ContrPair w.2.2 where
```

def bound[\[source\]](#)

The sum of `boundFst` and `boundSnd`, giving on `Sum.inl k` the first index in the `k`th contraction, and on `Sum.inr k` the second index in the `k`th contraction.

```
def bound {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) : Fin k ⊕ Fin k → Fin n
```

lemma bound_inl[\[source\]](#)

On Sum.inl k the map bound acts via boundFst .

```
@[simp]
lemma bound_inl {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) (i : Fin k) : w.bound (Sum.inl i) = w.boundFst i
```

lemma bound_inr[\[source\]](#)

On Sum.inr k the map bound acts via boundSnd .

```
@[simp]
lemma bound_inr {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) (i : Fin k) : w.bound (Sum.inr i) = w.boundSnd i
```

lemma bound_injection[\[source\]](#)

```
lemma bound_injection {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) : Function.Injective w.bound
```

lemma bound_le_total[\[source\]](#)

```
lemma bound_le_total {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) : 2 * k ≤ n
```

def unboundList[\[source\]](#)

The list of fields (indexed by $\text{Fin } n$) in a Wick contraction which are not bound, i.e. which do not appear in any contraction.

```
def unboundList {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) : List (Fin n)
```

lemma unboundList_nodup[\[source\]](#)

The list of field positions which are not contracted has no duplicates.

```
lemma unboundList_nodup {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) : (w.unboundList).Nodup
```

lemma unboundList_length[\[source\]](#)

*The length of the unboundList is equal to $n - 2 * k$. That is the total number of fields minus the number of contracted fields.*

```
lemma unboundList_length {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} (w : WickContract str b1 b2) :
  w.unboundList.length = n - 2 * k
```

lemma unboundList_sorted[\[source\]](#)

```
lemma unboundList_sorted {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n} (w : WickContract str b1 b2) :
  List.Sorted (fun i j => i < j) w.unboundList
```

def unbound[\[source\]](#)

The ordered embedding giving the fields which are not bound in a contraction. These are the fields that will appear in a normal operator in Wick's theorem.

```
def unbound {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ}
  {no : ℕ} {o : Fin no → ℰ} {str : WickString i c o final}
  {k : ℕ} {b1 b2 : Fin k → Fin n}
  (w : WickContract str b1 b2) : Fin (n - 2 * k) → Fin n where
```

informal_definition HasEqualTimeContractions[\[source\]](#)

```
informal_definition HasEqualTimeContractions where
  math := "The condition for a Wick contraction to have two fields contracted
  which are of equal time, i.e. come from the same vertex."
  deps := [``WickContract]
```

2.5 HepLean.FeynmanDiagrams.Wick.Species

[HepLean/FeynmanDiagrams/Wick/Species.lean](#) on GitHub.

structure Species[\[source\]](#)

The basic structure needed to write down Wick contractions for a theory and calculate the corresponding Feynman diagrams.

WARNING: This definition is not yet complete.

```
structure Species where
  ℱ : Type
  ξ : ℱ → ℱ
  ξ_involutive : Function.Involutive ξ
  ℳ : Type
  ℳFields : ℳ → Σ n, Fin n → ℱ
```

2.6 HepLean.FeynmanDiagrams.Wick.String

[HepLean/FeynmanDiagrams/Wick/String.lean](#) on GitHub.

inductive $\mathcal{E}$ [\[source\]](#)

The colors of edges which one can associate with a vertex for a theory with two complex scalar fields.

inductive $\mathcal{E}$ where

```
| complexScalarOut1 :  $\mathcal{E}$ 
| complexScalarIn1 :  $\mathcal{E}$ 
| complexScalarOut2 :  $\mathcal{E}$ 
| complexScalarIn2 :  $\mathcal{E}$ 
```

def ξ [\[source\]](#)

The map taking each color to it's dual, specifying how we can contract edges.

def ξ : $\mathcal{E} \rightarrow \mathcal{E}$

```
|  $\mathcal{E}.complexScalarOut_1 \Rightarrow \mathcal{E}.complexScalarIn_1$ 
|  $\mathcal{E}.complexScalarIn_1 \Rightarrow \mathcal{E}.complexScalarOut_1$ 
|  $\mathcal{E}.complexScalarOut_2 \Rightarrow \mathcal{E}.complexScalarIn_2$ 
|  $\mathcal{E}.complexScalarIn_2 \Rightarrow \mathcal{E}.complexScalarOut_2$ 
```

lemma $\xi_involutive$ [\[source\]](#)

The function ξ is an involution.

lemma $\xi_involutive$: Function.Involutive ξ

inductive $\mathcal{V}$ [\[source\]](#)

The vertices associated with two complex scalars. We call this type, the type of vertex colors.

inductive $\mathcal{V}$ where

```
|  $\phi_1 \phi_1 \phi_2 \phi_2$  :  $\mathcal{V}$ 
|  $\phi_1 \phi_1 \phi_1 \phi_1$  :  $\mathcal{V}$ 
|  $\phi_2 \phi_2 \phi_2 \phi_2$  :  $\mathcal{V}$ 
```

def $\mathcal{V}NoEdges$ [\[source\]](#)

To each vertex, the association of the number of edges.

@[no!int unusedArguments]

def $\mathcal{V}NoEdges$: $\mathcal{V} \rightarrow \mathbb{N}$

def $\mathcal{V}Edges$ [\[source\]](#)

To each vertex, associates the indexing map of half-edges associated with that edge.

def $\mathcal{V}Edges$ (v : $\mathcal{V}$) : Fin ($\mathcal{V}NoEdges$ v) $\rightarrow \mathcal{E}$

inductive WickStringLast[\[source\]](#)

A helper function for WickString. It is used to seperate incoming, vertex, and outgoing nodes.

```

inductive WickStringLast where
  | incoming : WickStringLast
  | vertex : WickStringLast
  | outgoing : WickStringLast
  | final : WickStringLast

```

inductive WickString

[\[source\]](#)

A wick string is a representation of a string of fields from a theory. E.g. $\phi(x1) \phi(x2) \phi(y) \phi(y) \phi(y) \phi(x3)$. The use of vertices in the Wick string allows us to identify which fields have the same space-time coordinate.

Note: Fields are added to c from right to left - matching how we would write this on pen and paper.

```

inductive WickString : {ni : ℕ} → (i : Fin ni → ℰ) → {n : ℕ} → (c : Fin n → ℰ) →
  {no : ℕ} → (o : Fin no → ℰ) → WickStringLast → Type where
  | empty : WickString Fin.elim0 Fin.elim0 Fin.elim0 incoming
  | incoming {n ni no : ℕ} {i : Fin ni → ℰ} {c : Fin n → ℰ}
    {o : Fin no → ℰ} (w : WickString i c o incoming) (e : ℰ) :
    WickString (Fin.cons e i) (Fin.cons e c) o incoming
  | endIncoming {n ni no : ℕ} {i : Fin ni → ℰ} {c : Fin n → ℰ}
    {o : Fin no → ℰ} (w : WickString i c o incoming) : WickString i c o vertex
  | vertex {n ni no : ℕ} {i : Fin ni → ℰ} {c : Fin n → ℰ}
    {o : Fin no → ℰ} (w : WickString i c o vertex) (v : ℱ) :
    WickString i (Fin.append (ℱEdges v) c) o vertex
  | endVertex {n ni no : ℕ} {i : Fin ni → ℰ} {c : Fin n → ℰ}
    {o : Fin no → ℰ} (w : WickString i c o vertex) : WickString i c o outgoing
  | outgoing {n ni no : ℕ} {i : Fin ni → ℰ} {c : Fin n → ℰ}
    {o : Fin no → ℰ} (w : WickString i c o outgoing) (e : ℰ) :
    WickString i (Fin.cons e c) (Fin.cons e o) outgoing
  | endOutgoing {n ni no : ℕ} {i : Fin ni → ℰ} {c : Fin n → ℰ}
    {o : Fin no → ℰ} (w : WickString i c o outgoing) : WickString i c o final

```

def size

[\[source\]](#)

The number of nodes in a Wick string. This is used to help prove termination.

```

def size {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ} {no : ℕ} {o : Fin no → ℰ}
  {f : WickStringLast} : WickString i c o f → ℕ

```

def numVertex

[\[source\]](#)

The number of vertices in a Wick string. This does NOT include external vertices.

```

def numVertex {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ} {no : ℕ} {o : Fin no → ℰ}
  {f : WickStringLast} : WickString i c o f → ℕ

```

def vertices

[\[source\]](#)

The vertices present in a Wick string. This does NOT include external vertices.

```

def vertices {ni : ℕ} {i : Fin ni → ℰ} {n : ℕ} {c : Fin n → ℰ} {no : ℕ} {o : Fin no → ℰ}
  {f : WickStringLast} : (w : WickString i c o f) → Fin w.numVertex → ℱ

```

informal_definition fieldToVertex[\[source\]](#)

```
informal_definition fieldToVertex where
  math := "A function which takes a field and returns the vertex it is associated with.
    This is a map from `Fin n` to `Fin w.numVertex`"
  deps := [``WickString]
```

informal_definition exponentialPrefactor[\[source\]](#)

```
informal_definition exponentialPrefactor where
  math := "The combinatorial prefactor from the expansion of the
    exponential associated with a Wick string."
  deps := [``vertices, ``WickString]
```

informal_definition vertexPrefactor[\[source\]](#)

```
informal_definition vertexPrefactor where
  math := "The prefactor arising from the coefficient of vertices in the
    Lagrangian. This should not take account of the exponential prefactor."
  deps := [``vertices, ``WickString]
```

informal_definition minNoLoops[\[source\]](#)

```
informal_definition minNoLoops where
  math := "The minimum number of loops a Feynman diagram based on a given Wick string can have.
    There should be a lemma proving this statement."
  deps := [``WickString]
```

informal_definition LoopLevel[\[source\]](#)

```
informal_definition LoopLevel where
  math := "The type of Wick strings for fixed input and output which may permit a Feynman
    diagram
    which have a number of loops less than or equal to some number."
  deps := [``minNoLoops, ``WickString]
```

informal_lemma loopLevel_fintype[\[source\]](#)

```
informal_lemma loopLevel_fintype where
  math := "The instance of a finite type on `LoopLevel`."
  deps := [``LoopLevel]
```

2.7 HepLean.Lorentz.Algebra.Basic

[HepLean/Lorentz/Algebra/Basic.lean](#) on GitHub.

def lorentzAlgebra[\[source\]](#)

The Lorentz algebra as a subalgebra of Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ.

```
def lorentzAlgebra : LieSubalgebra ℝ (Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ)
```

lemma transpose_eta[\[source\]](#)

```
lemma transpose_eta (A : lorentzAlgebra) : A.1T * η = - η * A.1
```

lemma mem_of_transpose_eta_eq_eta_mul_self[\[source\]](#)

```
lemma mem_of_transpose_eta_eq_eta_mul_self {A : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ}
  (h : AT * η = - η * A) : A ∈ lorentzAlgebra
```

lemma mem_iff[\[source\]](#)

```
lemma mem_iff {A : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ} :
  A ∈ lorentzAlgebra ↔ AT * η = - η * A
```

lemma mem_iff'[\[source\]](#)

```
lemma mem_iff' (A : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ) :
  A ∈ lorentzAlgebra ↔ A = - η * AT * η
```

lemma diag_comp[\[source\]](#)

```
lemma diag_comp (Λ : lorentzAlgebra) (μ : Fin 1 ⊕ Fin 3) : Λ.1 μ μ = 0
```

lemma time_comps[\[source\]](#)

```
lemma time_comps (Λ : lorentzAlgebra) (i : Fin 3) :
  Λ.1 (Sum.inr i) (Sum.inl 0) = Λ.1 (Sum.inl 0) (Sum.inr i)
```

lemma space_comps[\[source\]](#)

```
lemma space_comps (Λ : lorentzAlgebra) (i j : Fin 3) :
  Λ.1 (Sum.inr i) (Sum.inr j) = - Λ.1 (Sum.inr j) (Sum.inr i)
```

2.8 HepLean.Lorentz.ComplexTensor.Basis

[HepLean/Lorentz/ComplexTensor/Basis.lean](#) on GitHub.

lemma basis_eq_FD[\[source\]](#)

```
lemma basis_eq_FD {n : ℕ} (c : Fin n → complexLorentzTensor.C)
  (b : Π j, Fin (complexLorentzTensor.repDim (c j))) (i : Fin n)
  (h : { as := c i } = { as := c1 }) :
  (complexLorentzTensor.FD.map (eqToHom h)).hom
  (complexLorentzTensor.basis (c i) (b i)) =
  (complexLorentzTensor.basis c1 (Fin.cast (by simp_all) (b i)))
```

2.9 HepLean.Lorentz.Group.Basic

[HepLean/Lorentz/Group/Basic.lean](#) on GitHub.

instance Invertible M.1[\[source\]](#)

The underlying matrix of a Lorentz transformation is invertible.

```
instance (M : LorentzGroup d) : Invertible M.1 where
```

lemma transpose_val[\[source\]](#)

```
lemma transpose_val : (transpose  $\Lambda$ ).1 =  $\Lambda.1^T$ 
```

lemma transpose_inv[\[source\]](#)

```
lemma transpose_inv : (transpose  $\Lambda$ )-1 = transpose  $\Lambda^{-1}$ 
```

lemma comm_minkowskiMatrix[\[source\]](#)

```
lemma comm_minkowskiMatrix :  $\Lambda.1$  * minkowskiMatrix = minkowskiMatrix * (transpose  $\Lambda^{-1}$ ).1
```

lemma minkowskiMatrix_comm[\[source\]](#)

```
lemma minkowskiMatrix_comm : minkowskiMatrix *  $\Lambda.1$  = (transpose  $\Lambda^{-1}$ ).1 * minkowskiMatrix
```

def parity[\[source\]](#)

The parity transformation.

```
def parity : LorentzGroup d
```

2.10 HepLean.Lorentz.Group.Boosts

[HepLean/Lorentz/Group/Boosts.lean](#) on GitHub.

lemma genBoostAux2_self[\[source\]](#)

```
lemma genBoostAux2_self (u : FuturePointing d) : genBoostAux2 u u = - genBoostAux1 u u
```

lemma genBoost_mul_one_plus_contr[\[source\]](#)

```
lemma genBoost_mul_one_plus_contr (u v : FuturePointing d) (x : Contr d) :
  (1 +  $\langle\langle u.1.1, v.1.1 \rangle\rangle_m$ ) • genBoost u v x =
  (1 +  $\langle\langle u.1.1, v.1.1 \rangle\rangle_m$ ) • x +
  (2 *  $\langle\langle x, u.val.val \rangle\rangle_m$  * (1 +  $\langle\langle u.1.1, v.1.1 \rangle\rangle_m$ )) • v.1.1
  - ( $\langle\langle x, u.1.1 + v.1.1 \rangle\rangle_m$ ) • (u.1.1 + v.1.1)
```

2.11 HepLean.Lorentz.MinkowskiMatrix

[HepLean/Lorentz/MinkowskiMatrix.lean](#) on GitHub.

lemma mulVec_inl_0[\[source\]](#)

The time components of a vector acted on by the Minkowski matrix remains unchanged.

```
@[simp]
lemma mulVec_inl_0 (v : (Fin 1 ⊕ Fin d) → ℝ) :
  (η *v v) (Sum.inl 0) = v (Sum.inl 0)
```

lemma mulVec_inr_i

[\[source\]](#)

The space components of a vector acted on by the Minkowski matrix swaps sign.

```
@[simp]
lemma mulVec_inr_i (v : (Fin 1 ⊕ Fin d) → ℝ) (i : Fin d) :
  (η *v v) (Sum.inr i) = - v (Sum.inr i)
```

def dual

[\[source\]](#)

The dual of a matrix with respect to the Minkowski metric. A suitable name for this construction is the Minkowski dual.

```
def dual : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma dual_id

[\[source\]](#)

The Minkowski dual of the identity is the identity.

```
@[simp]
lemma dual_id : @dual d 1 = 1
```

lemma dual_mul

[\[source\]](#)

The Minkowski dual swaps multiplications (acts contravariantly).

```
@[simp]
lemma dual_mul : dual (Λ * Λ') = dual Λ' * dual Λ
```

lemma dual_dual

[\[source\]](#)

The Minkowski dual is involutive (i.e. $\text{dual} (\text{dual } \Lambda) = \Lambda$).

```
@[simp]
lemma dual_dual : Function.Involutive (@dual d)
```

lemma dual_eta

[\[source\]](#)

The Minkowski dual preserves the Minkowski matrix.

```
@[simp]
lemma dual_eta : @dual d η = η
```

lemma dual_transpose

[\[source\]](#)

The Minkowski dual commutes with the transpose.

```
@[simp]
lemma dual_transpose : dual ΛT = (dual Λ)T
```

lemma det_dual[\[source\]](#)

The Minkowski dual preserves determinants.

```
@[simp]
lemma det_dual : (dual  $\Lambda$ ).det =  $\Lambda$ .det
```

lemma dual_apply[\[source\]](#)

Expansion of the components of the Minkowski dual in terms of the components of the original matrix.

```
lemma dual_apply ( $\mu$   $\nu$  : Fin 1  $\otimes$  Fin d) :
  dual  $\Lambda$   $\mu$   $\nu$  =  $\eta$   $\mu$   $\mu$  *  $\Lambda$   $\nu$   $\mu$  *  $\eta$   $\nu$   $\nu$ 
```

lemma dual_apply_minkowskiMatrix[\[source\]](#)

The components of the Minkowski dual of a matrix multiplied by the Minkowski matrix in terms of the original matrix.

```
lemma dual_apply_minkowskiMatrix ( $\mu$   $\nu$  : Fin 1  $\otimes$  Fin d) :
  dual  $\Lambda$   $\mu$   $\nu$  *  $\eta$   $\nu$   $\nu$  =  $\eta$   $\mu$   $\mu$  *  $\Lambda$   $\nu$   $\mu$ 
```

2.12 HepLean.Lorentz.RealVector.Basic

[HepLean/Lorentz/RealVector/Basic.lean](#) on GitHub.

def Contr[\[source\]](#)

The representation of LorentzGroup d on real vectors corresponding to contravariant Lorentz vectors. In index notation these have an up index ψ^i .

```
def Contr (d :  $\mathbb{N}$ ) : Rep  $\mathbb{R}$  (LorentzGroup d)
```

instance TopologicalSpace (Contr d)[\[source\]](#)

The representation of contravariant Lorentz vectors forms a topological space, induced by its equivalence to Fin 1 $\otimes$ Fin d $\rightarrow \mathbb{R}$.

```
instance : TopologicalSpace (Contr d)
```

lemma continuous_contr[\[source\]](#)

```
lemma continuous_contr {T : Type} [TopologicalSpace T] (f : T  $\rightarrow$  Contr d)
  (h : Continuous ( $\text{fun } i \Rightarrow (f i).toFinId \mathbb{R}$ )) : Continuous f
```

lemma contr_continuous[\[source\]](#)

```
lemma contr_continuous {T : Type} [TopologicalSpace T] (f : Contr d  $\rightarrow$  T)
  (h : Continuous (f  $\circ$  (@ContrMod.toFinId  $\mathbb{R}$ Equiv d).symm)) : Continuous f
```

def Co [source]

The representation of LorentzGroup d on real vectors corresponding to covariant Lorentz vectors. In index notation these have an up index ψ^i .

```
def Co (d : ℕ) : Rep ℝ (LorentzGroup d)
```

def toCo [source]

The morphism of representations from Contr d to Co d defined by multiplication with the metric.

```
def Contr.toCo (d : ℕ) : Contr d → Co d where
```

def toContr [source]

The morphism of representations from Co d to Contr d defined by multiplication with the metric.

```
def Co.toContr (d : ℕ) : Co d → Contr d where
```

def contrIsoCo [source]

The isomorphism between Contr d and Co d induced by multiplication with the Minkowski metric.

```
def contrIsoCo (d : ℕ) : Contr d ≅ Co d where
```

lemma p_stdBasis [source]

```
lemma p_stdBasis (μ : Fin 1 ⊕ Fin 3) (Λ : LorentzGroup 3) :  
  (Contr 3).p Λ (ContrMod.stdBasis μ) = ∑ j, Λ.1 j μ • ContrMod.stdBasis j
```

2.13 HepLean.Lorentz.RealVector.Contraction

[HepLean/Lorentz/RealVector/Contraction.lean](#) on GitHub.

def contrModCoModBi [source]

The bi-linear map corresponding to contraction of a contravariant Lorentz vector with a covariant Lorentz vector.

```
def contrModCoModBi (d : ℕ) : ContrMod d →ℓ[ℝ] CoMod d →ℓ[ℝ] ℝ where
```

def coModContrModBi [source]

The bi-linear map corresponding to contraction of a covariant Lorentz vector with a contravariant Lorentz vector.

```
def coModContrModBi (d : ℕ) : CoMod d →ℓ[ℝ] ContrMod d →ℓ[ℝ] ℝ where
```

def contrCoContract[\[source\]](#)

The linear map from $\text{Contr } d \otimes \text{Co } d$ to $\mathbb{R}$ given by summing over components of contravariant Lorentz vector and covariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\psi^i \varphi_i$.

```
def contrCoContract : Contr d ⊗ Co d → ℔_ (Rep ℝ (LorentzGroup d)) where
```

lemma contrCoContract_hom_tmul[\[source\]](#)

```
lemma contrCoContract_hom_tmul (ψ : Contr d) (φ : Co d) : ⟨⟨ψ, φ⟩⟩_m = ψ.toFinldℝ ·_v φ.toFinldℝ
```

def coContrContract[\[source\]](#)

The linear map from $\text{Co } d \otimes \text{Contr } d$ to $\mathbb{R}$ given by summing over components of contravariant Lorentz vector and covariant Lorentz vector in the standard basis (i.e. the dot product). In terms of index notation this is the contraction is $\psi^i \varphi_i$.

```
def coContrContract : Co d ⊗ Contr d → ℔_ (Rep ℝ (LorentzGroup d)) where
```

lemma coContrContract_hom_tmul[\[source\]](#)

```
lemma coContrContract_hom_tmul (φ : Co d) (ψ : Contr d) : ⟨⟨φ, ψ⟩⟩_m = φ.toFinldℝ ·_v ψ.toFinldℝ
```

lemma contrCoContract_tmul_symm[\[source\]](#)

```
lemma contrCoContract_tmul_symm (φ : Contr d) (ψ : Co d) : ⟨⟨φ, ψ⟩⟩_m = ⟨⟨ψ, φ⟩⟩_m
```

lemma coContrContract_tmul_symm[\[source\]](#)

```
lemma coContrContract_tmul_symm (φ : Co d) (ψ : Contr d) : ⟨⟨φ, ψ⟩⟩_m = ⟨⟨ψ, φ⟩⟩_m
```

def contrContrContract[\[source\]](#)

The linear map from $\text{Contr } d \otimes \text{Contr } d$ to $\mathbb{R}$ induced by the homomorphism Contr.toCo and the contraction contrCoContract .

```
def contrContrContract : Contr d ⊗ Contr d → ℔_ (Rep ℝ (LorentzGroup d))
```

def contrContrContractField[\[source\]](#)

The linear map from $\text{Contr } d \otimes \text{Contr } d$ to $\mathbb{R}$ induced by the homomorphism Contr.toCo and the contraction contrCoContract .

```
def contrContrContractField : (Contr d).V ⊗[℔] (Contr d).V →_l[℔] ℝ
```

lemma contrContrContract_hom_tmul[\[source\]](#)

```
lemma contrContrContract_hom_tmul (φ : Contr d) (ψ : Contr d) :
  ⟨⟨φ, ψ⟩⟩_m = φ.toFinldℝ ·_v η *_v ψ.toFinldℝ
```

def coCoContract[\[source\]](#)

The linear map from $Co\ d \otimes Co\ d$ to $\mathbb{R}$ induced by the homomorphism $Co.toContr$ and the contraction $coContrContract$.

```
def coCoContract : Co d ⊗ Co d → ℝ (Rep ℝ (LorentzGroup d))
```

lemma coCoContract_hom_tmul[\[source\]](#)

```
lemma coCoContract_hom_tmul (φ : Co d) (ψ : Co d) :
  ⟨⟨φ, ψ⟩⟩m = φ.toFinldℝ ·v η*v ψ.toFinldℝ
```

lemma action_tmul[\[source\]](#)

```
@[simp]
lemma action_tmul (g : LorentzGroup d) : ⟨⟨(Contr d).p g x, (Contr d).p g y⟩⟩m = ⟨⟨x, y⟩⟩m
```

lemma as_sum[\[source\]](#)

```
lemma as_sum : ⟨⟨x, y⟩⟩m = x.val (Sum.inl 0) * y.val (Sum.inl 0) -
  ∑ i, x.val (Sum.inr i) * y.val (Sum.inr i)
```

lemma as_sum_toSpace[\[source\]](#)

```
lemma as_sum_toSpace : ⟨⟨x, y⟩⟩m = x.val (Sum.inl 0) * y.val (Sum.inl 0) -
  ⟨⟨x.toSpace, y.toSpace⟩⟩ℝ
```

lemma stdBasis_inl[\[source\]](#)

```
lemma stdBasis_inl {d : ℕ} :
  ⟨⟨@ContrMod.stdBasis d (Sum.inl 0), ContrMod.stdBasis (Sum.inl 0)⟩⟩m = (1 : ℝ)
```

lemma symm[\[source\]](#)

```
lemma symm : ⟨⟨x, y⟩⟩m = ⟨⟨y, x⟩⟩m
```

lemma dual_mulVec_right[\[source\]](#)

```
lemma dual_mulVec_right : ⟨⟨x, dual ∧*v y⟩⟩m = ⟨⟨∧*v x, y⟩⟩m
```

lemma dual_mulVec_left[\[source\]](#)

```
lemma dual_mulVec_left : ⟨⟨dual ∧*v x, y⟩⟩m = ⟨⟨x, ∧*v y⟩⟩m
```

lemma right_parity[\[source\]](#)

```
lemma right_parity : ⟨⟨x, (Contr d).p LorentzGroup.parity y⟩⟩m = ∑ i, x.val i * y.val i
```

lemma self_parity_eq_zero_iff[\[source\]](#)

lemma self_parity_eq_zero_iff : $\langle\langle y, (\text{Contr } d).p \text{ LorentzGroup.parity } y \rangle\rangle_m = 0 \leftrightarrow y = 0$

lemma nondegenerate[\[source\]](#)

The metric tensor is non-degenerate.

lemma nondegenerate : $(\forall (x : \text{Contr } d), \langle\langle x, y \rangle\rangle_m = 0) \leftrightarrow y = 0$

lemma matrix_apply_eq_iff_sub[\[source\]](#)

lemma matrix_apply_eq_iff_sub : $\langle\langle x, \Lambda *_v y \rangle\rangle_m = \langle\langle x, \Lambda' *_v y \rangle\rangle_m \leftrightarrow \langle\langle x, (\Lambda - \Lambda') *_v y \rangle\rangle_m = 0$

lemma matrix_eq_iff_eq_forall'[\[source\]](#)

lemma matrix_eq_iff_eq_forall' : $(\forall (v : \text{Contr } d), (\Lambda *_v v) = \Lambda' *_v v) \leftrightarrow$
 $\forall (w v : \text{Contr } d), \langle\langle v, \Lambda *_v w \rangle\rangle_m = \langle\langle v, \Lambda' *_v w \rangle\rangle_m$

lemma matrix_eq_iff_eq_forall[\[source\]](#)

lemma matrix_eq_iff_eq_forall : $\Lambda = \Lambda' \leftrightarrow \forall (w v : \text{Contr } d), \langle\langle v, \Lambda *_v w \rangle\rangle_m = \langle\langle v, \Lambda' *_v w \rangle\rangle_m$

lemma matrix_eq_id_iff[\[source\]](#)

lemma matrix_eq_id_iff : $\Lambda = 1 \leftrightarrow \forall (w v : \text{Contr } d), \langle\langle v, \Lambda *_v w \rangle\rangle_m = \langle\langle v, w \rangle\rangle_m$

lemma mem_iff_invariant[\[source\]](#)

lemma _root_.LorentzGroup.mem_iff_invariant : $\Lambda \in \text{LorentzGroup } d \leftrightarrow$
 $\forall (w v : \text{Contr } d), \langle\langle \Lambda *_v v, \Lambda *_v w \rangle\rangle_m = \langle\langle v, w \rangle\rangle_m$

lemma inl_sq_eq[\[source\]](#)

lemma inl_sq_eq (v : Contr d) : $v.\text{val } (\text{Sum.inl } 0) ^ 2 =$
 $(\langle\langle v, v \rangle\rangle_m) + \sum i, v.\text{val } (\text{Sum.inr } i) ^ 2$

lemma le_inl_sq[\[source\]](#)

lemma le_inl_sq (v : Contr d) : $\langle\langle v, v \rangle\rangle_m \leq v.\text{val } (\text{Sum.inl } 0) ^ 2$

lemma ge_abs_inner_product[\[source\]](#)

lemma ge_abs_inner_product (v w : Contr d) : $v.\text{val } (\text{Sum.inl } 0) * w.\text{val } (\text{Sum.inl } 0) -$
 $\|\langle\langle v.\text{toSpace}, w.\text{toSpace} \rangle\rangle_{\mathbb{R}}\| \leq \langle\langle v, w \rangle\rangle_m$

lemma ge_sub_norm[\[source\]](#)

lemma ge_sub_norm (v w : Contr d) : $v.\text{val } (\text{Sum.inl } 0) * w.\text{val } (\text{Sum.inl } 0) -$
 $\|v.\text{toSpace}\| * \|w.\text{toSpace}\| \leq \langle\langle v, w \rangle\rangle_m$

lemma basis_left[\[source\]](#)

```
@[simp]
lemma basis_left {v : Contr d} (μ : Fin 1 ⊕ Fin d) :
  ⟨⟨ContrMod.stdBasis μ, v⟩⟩m = η μ μ * v.toFin1dℝ μ
```

lemma on_basis_mulVec[\[source\]](#)

```
lemma on_basis_mulVec (μ v : Fin 1 ⊕ Fin d) :
  ⟨⟨ContrMod.stdBasis μ, Λ *v ContrMod.stdBasis v⟩⟩m = η μ μ * Λ μ v
```

lemma on_basis[\[source\]](#)

```
lemma on_basis (μ v : Fin 1 ⊕ Fin d) : ⟨⟨ContrMod.stdBasis μ, ContrMod.stdBasis v⟩⟩m = η μ v
```

lemma matrix_apply_stdBasis[\[source\]](#)

```
lemma matrix_apply_stdBasis (v μ : Fin 1 ⊕ Fin d) :
  Λ v μ = η v v * ⟨⟨ContrMod.stdBasis v, Λ *v ContrMod.stdBasis μ⟩⟩m
```

lemma same_eq_det_toSelfAdjoint[\[source\]](#)

```
lemma same_eq_det_toSelfAdjoint (x : ContrMod 3) :
  ⟨⟨x, x⟩⟩m = det (ContrMod.toSelfAdjoint x).1
```

2.14 HepLean.Lorentz.RealVector.Modules

[HepLean/Lorentz/RealVector/Modules.lean](#) on GitHub.

structure ContrMod[\[source\]](#)

The module for contravariant (up-index) real Lorentz vectors.

```
structure ContrMod (d : ℕ) where
  val : Fin 1 ⊕ Fin d → ℝ
```

lemma ContrMod.ext[\[source\]](#)

```
@[ext]
lemma ext {ψ ψ' : ContrMod d} (h : ψ.val = ψ'.val) : ψ = ψ'
```

def ContrMod.toFin1dℝFun[\[source\]](#)

The equivalence between ContrModule and Fin 1 ⊕ Fin d → ℝ.

```
def toFin1dℝFun : ContrMod d ≈ (Fin 1 ⊕ Fin d → ℝ) where
```

instance AddCommMonoid (ContrMod d)[\[source\]](#)

The instance of AddCommMonoid on ContrModule defined via its equivalence with Fin 1 ⊕ Fin d → ℝ.

```
instance : AddCommMonoid (ContrMod d)
```

instance AddCommGroup (ContrMod d)[\[source\]](#)

The instance of AddCommGroup on Contr $\mathbb{R}$ Module defined via its equivalence with Fin 1 $\otimes$ Fin d $\rightarrow \mathbb{R}$.

```
instance : AddCommGroup (ContrMod d)
```

instance Module $\mathbb{R}$ (ContrMod d)[\[source\]](#)

The instance of Module on Contr $\mathbb{R}$ Module defined via its equivalence with Fin 1 $\otimes$ Fin d $\rightarrow \mathbb{R}$.

```
instance : Module  $\mathbb{R}$  (ContrMod d)
```

lemma val_add[\[source\]](#)

```
@[simp]
```

```
lemma val_add ( $\psi \ \psi'$  : ContrMod d) : ( $\psi + \psi'$ ).val =  $\psi$ .val +  $\psi'$ .val
```

lemma val_smul[\[source\]](#)

```
@[simp]
```

```
lemma val_smul ( $r$  :  $\mathbb{R}$ ) ( $\psi$  : ContrMod d) : ( $r \cdot \psi$ ).val =  $r \cdot \psi$ .val
```

def ContrMod.toFinIdREquiv[\[source\]](#)

The linear equivalence between Contr $\mathbb{R}$ Module and (Fin 1 $\otimes$ Fin d $\rightarrow \mathbb{R}$).

```
def toFinIdREquiv : ContrMod d  $\approx_l[\mathbb{R}]$  (Fin 1  $\otimes$  Fin d  $\rightarrow \mathbb{R}$ )
```

abbrev ContrMod.toFinId $\mathbb{R}$ [\[source\]](#)

The underlying element of Fin 1 $\otimes$ Fin d $\rightarrow \mathbb{R}$ of a element in Contr $\mathbb{R}$ Module defined through the linear equivalence toFinIdREquiv.

```
abbrev toFinId $\mathbb{R}$  ( $\psi$  : ContrMod d)
```

lemma toFinId $\mathbb{R}$ _eq_val[\[source\]](#)

```
lemma toFinId $\mathbb{R}$ _eq_val ( $\psi$  : ContrMod d) :  $\psi$ .toFinId $\mathbb{R}$  =  $\psi$ .val
```

def ContrMod.stdBasis[\[source\]](#)

The standard basis of Contr $\mathbb{R}$ Module indexed by Fin 1 $\otimes$ Fin d.

```
@[simps!]
```

```
def stdBasis : Basis (Fin 1  $\otimes$  Fin d)  $\mathbb{R}$  (ContrMod d)
```

lemma ContrMod.stdBasis_toFinIdREquiv_apply_same[\[source\]](#)

```
@[simp]
```

```
lemma stdBasis_toFinIdREquiv_apply_same ( $\mu$  : Fin 1  $\otimes$  Fin d) :  
  toFinIdREquiv (stdBasis  $\mu$ )  $\mu$  = 1
```

lemma ContrMod.stdBasis_apply_same[\[source\]](#)

@[simp]

lemma stdBasis_apply_same (μ : $\text{Fin } 1 \otimes \text{Fin } d$) : (stdBasis μ).val μ = 1**lemma ContrMod.stdBasis_toFinldREquiv_apply_ne**[\[source\]](#)**lemma** stdBasis_toFinldREquiv_apply_ne { $\mu \ v$: $\text{Fin } 1 \otimes \text{Fin } d$ } (h : $\mu \neq v$) :
toFinldREquiv (stdBasis μ) v = 0**lemma stdBasis_inl_apply_inr**[\[source\]](#)

@[simp]

lemma stdBasis_inl_apply_inr (i : $\text{Fin } d$) : (stdBasis (Sum.inl 0)).val (Sum.inr i) = 0**lemma ContrMod.stdBasis_apply**[\[source\]](#)**lemma** stdBasis_apply ($\mu \ v$: $\text{Fin } 1 \otimes \text{Fin } d$) : (stdBasis μ).val v = **if** μ = v **then** 1 **else** 0**lemma ContrMod.stdBasis_decomp**[\[source\]](#)*Decomposition of a contrvariant Lorentz vector into the standard basis.***lemma** stdBasis_decomp (v : ContrMod d) : $v = \sum i, v.\text{toFinldR } i \cdot \text{stdBasis } i$ **abbrev ContrMod.mulVec**[\[source\]](#)*Multiplication of a matrix with a vector in ContrMod.***abbrev** mulVec (M : Matrix ($\text{Fin } 1 \otimes \text{Fin } d$) ($\text{Fin } 1 \otimes \text{Fin } d$) $\mathbb{R}$) (v : ContrMod d) :
ContrMod d **lemma ContrMod.mulVec_toFinldR**[\[source\]](#)

@[simp]

lemma mulVec_toFinldR (M : Matrix ($\text{Fin } 1 \otimes \text{Fin } d$) ($\text{Fin } 1 \otimes \text{Fin } d$) $\mathbb{R}$) (v : ContrMod d) :
($M *_v v$).toFinldR = $M *_v v.\text{toFinldR}$ **lemma mulVec_sub**[\[source\]](#)**lemma** mulVec_sub (M : Matrix ($\text{Fin } 1 \otimes \text{Fin } d$) ($\text{Fin } 1 \otimes \text{Fin } d$) $\mathbb{R}$) ($v \ w$: ContrMod d) :
 $M *_v (v - w) = M *_v v - M *_v w$ **lemma sub_mulVec**[\[source\]](#)**lemma** sub_mulVec ($M \ N$: Matrix ($\text{Fin } 1 \otimes \text{Fin } d$) ($\text{Fin } 1 \otimes \text{Fin } d$) $\mathbb{R}$) (v : ContrMod d) :
($M - N$) $*_v v = M *_v v - N *_v v$ **lemma mulVec_add**[\[source\]](#)**lemma** mulVec_add (M : Matrix ($\text{Fin } 1 \otimes \text{Fin } d$) ($\text{Fin } 1 \otimes \text{Fin } d$) $\mathbb{R}$) ($v \ w$: ContrMod d) :
 $M *_v (v + w) = M *_v v + M *_v w$

lemma one_mulVec[\[source\]](#)`@[simp]``lemma one_mulVec (v : ContrMod d) : (1 : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) *v v = v`**lemma mulVec\_mulVec**[\[source\]](#)`lemma mulVec_mulVec (M N : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) (v : ContrMod d) :
M *v (N *v v) = (M * N) *v v`**def norm**[\[source\]](#)

A NormedAddCommGroup structure on ContrMod. This is not an instance, as we don't want it to be applied always.

`def norm : NormedAddCommGroup (ContrMod d) where`**def toSpace**[\[source\]](#)

The underlying space part of a ContrMod formed by removing the first element. A better name for this might be tail.

`def toSpace (v : ContrMod d) : EuclideanSpace ℝ (Fin d)`**def ContrMod.rep**[\[source\]](#)

The representation of the Lorentz group acting on ContrℝModule d.

`def rep : Representation ℝ (LorentzGroup d) (ContrMod d) where`**lemma rep_apply_toFinldℝ**[\[source\]](#)`lemma rep_apply_toFinldℝ (g : LorentzGroup d) (ψ : ContrMod d) :
(rep g ψ).toFinldℝ = g.1 *v ψ.toFinldℝ`**def toSelfAdjoint**[\[source\]](#)

The linear equivalence between the vector-space ContrMod 3 and self-adjoint 2×2-complex matrices.

`def toSelfAdjoint : ContrMod 3 ≈ℓ[ℝ] selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)`**lemma toSelfAdjoint_apply**[\[source\]](#)`lemma toSelfAdjoint_apply (x : ContrMod 3) : toSelfAdjoint x =
x.toFinldℝ (Sum.inl 0) • ⟨PauliMatrix.σ0, PauliMatrix.σ0_selfAdjoint⟩
- x.toFinldℝ (Sum.inr 0) • ⟨PauliMatrix.σ1, PauliMatrix.σ1_selfAdjoint⟩
- x.toFinldℝ (Sum.inr 1) • ⟨PauliMatrix.σ2, PauliMatrix.σ2_selfAdjoint⟩
- x.toFinldℝ (Sum.inr 2) • ⟨PauliMatrix.σ3, PauliMatrix.σ3_selfAdjoint⟩`

lemma toSelfAdjoint_apply_coe[\[source\]](#)

```

lemma toSelfAdjoint_apply_coe (x : ContrMod 3) : (toSelfAdjoint x).1 =
  x.toFin1dℝ (Sum.inl 0) • PauliMatrix.σ0
- x.toFin1dℝ (Sum.inr 0) • PauliMatrix.σ1
- x.toFin1dℝ (Sum.inr 1) • PauliMatrix.σ2
- x.toFin1dℝ (Sum.inr 2) • PauliMatrix.σ3

```

lemma toSelfAdjoint_stdBasis[\[source\]](#)

```

lemma toSelfAdjoint_stdBasis (i : Fin 1 ⊗ Fin 3) :
  toSelfAdjoint (stdBasis i) = PauliMatrix.σSAL i

```

lemma toSelfAdjoint_symm_basis[\[source\]](#)

```

@[simp]
lemma toSelfAdjoint_symm_basis (i : Fin 1 ⊗ Fin 3) :
  toSelfAdjoint.symm (PauliMatrix.σSAL i) = (stdBasis i)

```

instance TopologicalSpace (ContrMod d)[\[source\]](#)

The type ContrMod d carries an instance of a topological group, induced by it's equivalence to $\text{Fin } 1 \otimes \text{Fin } d \rightarrow \mathbb{R}$.

```

instance : TopologicalSpace (ContrMod d)

```

lemma toFin1dℝEquiv_isInducing[\[source\]](#)

```

lemma toFin1dℝEquiv_isInducing : IsInducing (@ContrMod.toFin1dℝEquiv d)

```

lemma toFin1dℝEquiv_symm_isInducing[\[source\]](#)

```

lemma toFin1dℝEquiv_symm_isInducing : IsInducing ((@ContrMod.toFin1dℝEquiv d).symm)

```

structure CoMod[\[source\]](#)

The module for covariant (up-index) complex Lorentz vectors.

```

structure CoMod (d : ℕ) where
  val : Fin 1 ⊗ Fin d → ℝ

```

lemma CoMod.ext[\[source\]](#)

```

@[ext]
lemma ext {ψ ψ' : CoMod d} (h : ψ.val = ψ'.val) : ψ = ψ'

```

def CoMod.toFin1dℝFun[\[source\]](#)

The equivalence between CoModule and $\text{Fin } 1 \otimes \text{Fin } d \rightarrow \mathbb{R}$.

```

def toFin1dℝFun : CoMod d ≈ (Fin 1 ⊗ Fin d → ℝ) where

```

instance AddCommMonoid (CoMod d)[\[source\]](#)

The instance of AddCommMonoid on CoCModule defined via its equivalence with $\text{Fin } 1 \oplus \text{Fin } d \rightarrow \mathbb{R}$.

```
instance : AddCommMonoid (CoMod d)
```

instance AddCommGroup (CoMod d)[\[source\]](#)

The instance of AddCommGroup on CoRModule defined via its equivalence with $\text{Fin } 1 \oplus \text{Fin } d \rightarrow \mathbb{R}$.

```
instance : AddCommGroup (CoMod d)
```

instance Module ℝ (CoMod d)[\[source\]](#)

The instance of Module on CoRModule defined via its equivalence with $\text{Fin } 1 \oplus \text{Fin } d \rightarrow \mathbb{R}$.

```
instance : Module ℝ (CoMod d)
```

def CoMod.toFinldREquiv[\[source\]](#)

The linear equivalence between CoRModule and $(\text{Fin } 1 \oplus \text{Fin } d \rightarrow \mathbb{R})$.

```
def toFinldREquiv : CoMod d  $\approx_l$  [ℝ] (Fin 1  $\oplus$  Fin d  $\rightarrow$  ℝ)
```

abbrev CoMod.toFinldℝ[\[source\]](#)

The underlying element of $\text{Fin } 1 \oplus \text{Fin } d \rightarrow \mathbb{R}$ of a element in CoRModule defined through the linear equivalence toFinldREquiv.

```
abbrev toFinldℝ (ψ : CoMod d)
```

def CoMod.stdBasis[\[source\]](#)

The standard basis of CoRModule indexed by $\text{Fin } 1 \oplus \text{Fin } d$.

```
@[simps!]
```

```
def stdBasis : Basis (Fin 1  $\oplus$  Fin d) ℝ (CoMod d)
```

lemma CoMod.stdBasis_toFinldREquiv_apply_same[\[source\]](#)

```
@[simp]
```

```
lemma stdBasis_toFinldREquiv_apply_same (μ : Fin 1  $\oplus$  Fin d) :  
  toFinldREquiv (stdBasis μ) μ = 1
```

lemma CoMod.stdBasis_apply_same[\[source\]](#)

```
@[simp]
```

```
lemma stdBasis_apply_same (μ : Fin 1  $\oplus$  Fin d) : (stdBasis μ).val μ = 1
```

lemma CoMod.stdBasis_toFinIdℝEquiv_apply_ne[\[source\]](#)

```
lemma stdBasis_toFinIdℝEquiv_apply_ne {μ v : Fin 1 ⊗ Fin d} (h : μ ≠ v) :
  toFinIdℝEquiv (stdBasis μ) v = 0
```

lemma CoMod.stdBasis_apply[\[source\]](#)

```
lemma stdBasis_apply (μ v : Fin 1 ⊗ Fin d) : (stdBasis μ).val v = if μ = v then 1 else 0
```

lemma CoMod.stdBasis_decomp[\[source\]](#)

Decomposition of a covariant Lorentz vector into the standard basis.

```
lemma stdBasis_decomp (v : CoMod d) : v = ∑ i, v.toFinIdℝ i • stdBasis i
```

abbrev CoMod.mulVec[\[source\]](#)

Multiplication of a matrix with a vector in CoMod.

```
abbrev mulVec (M : Matrix (Fin 1 ⊗ Fin d) (Fin 1 ⊗ Fin d) ℝ) (v : CoMod d) :
  CoMod d
```

lemma CoMod.mulVec_toFinIdℝ[\[source\]](#)

```
@[simp]
lemma mulVec_toFinIdℝ (M : Matrix (Fin 1 ⊗ Fin d) (Fin 1 ⊗ Fin d) ℝ) (v : CoMod d) :
  (M *v v).toFinIdℝ = M *v v.toFinIdℝ
```

def CoMod.rep[\[source\]](#)

The representation of the Lorentz group acting on CoModule d.

```
def rep : Representation ℝ (LorentzGroup d) (CoMod d) where
```

2.15 HepLean.Lorentz.RealVector.NormOne

[HepLean/Lorentz/RealVector/NormOne.lean](#) on GitHub.

def NormOne[\[source\]](#)

The set of Lorentz vectors with norm 1.

```
def NormOne (d : ℕ) : Set (Contr d)
```

lemma NormOne.mem_iff[\[source\]](#)

```
lemma mem_iff {v : Contr d} : v ∈ NormOne d ↔ ⟨⟨v, v⟩⟩m = (1 : ℝ)
```

lemma contr_self[\[source\]](#)

```
@[simp]
lemma contr_self (v : NormOne d) : ⟨⟨v.1, v.1⟩⟩m = (1 : ℝ)
```

lemma mem_mulAction[\[source\]](#)

```
lemma mem_mulAction (g : LorentzGroup d) (v : Contr d) :
  v ∈ NormOne d ↔ (Contr d).p g v ∈ NormOne d
```

instance TopologicalSpace (NormOne d)[\[source\]](#)

The instance of a topological space on NormOne d defined as the subspace topology.

```
instance : TopologicalSpace (NormOne d)
```

def neg[\[source\]](#)

The negative of a NormOne as a NormOne.

```
def neg : NormOne d
```

def toNormOne[\[source\]](#)

The first column of a Lorentz matrix as a NormOneLorentzVector.

```
@[simps!]
def _root_.LorentzGroup.toNormOne (Λ : LorentzGroup d) : NormOne d
```

lemma toNormOne_inl[\[source\]](#)

```
lemma _root_.LorentzGroup.toNormOne_inl (Λ : LorentzGroup d) :
  (LorentzGroup.toNormOne Λ).val.val (Sum.inl 0) = Λ.1 (Sum.inl 0) (Sum.inl 0)
```

lemma toNormOne_inr[\[source\]](#)

```
lemma _root_.LorentzGroup.toNormOne_inr (Λ : LorentzGroup d) (i : Fin d) :
  (LorentzGroup.toNormOne Λ).val.val (Sum.inr i) = Λ.1 (Sum.inr i) (Sum.inl 0)
```

lemma inl_inl_mul[\[source\]](#)

```
lemma _root_.LorentzGroup.inl_inl_mul (Λ Λ' : LorentzGroup d) : (Λ * Λ').1 (Sum.inl 0) (Sum.inl 0) =
  ⟨⟨(LorentzGroup.toNormOne (LorentzGroup.transpose Λ)).1,
    (Contr d).p LorentzGroup.parity (LorentzGroup.toNormOne Λ').1⟩⟩m
```

lemma inl_sq[\[source\]](#)

```
lemma inl_sq : v.val.val (Sum.inl 0) ^ 2 = 1 + ||ContrMod.toSpace v.val|| ^ 2
```

lemma one_le_abs_inl[\[source\]](#)

```
lemma one_le_abs_inl : 1 ≤ |v.val.val (Sum.inl 0)|
```

lemma inl_le_neg_one_or_one_le_inl[\[source\]](#)

```
lemma inl_le_neg_one_or_one_le_inl : v.val.val (Sum.inl 0) ≤ -1 ∨ 1 ≤ v.val.val (Sum.inl 0)
```

lemma norm_space_le_abs_inl[\[source\]](#)

```
lemma norm_space_le_abs_inl : ||v.1.toSpace|| < |v.val.val (Sum.inl 0)|
```

lemma norm_space_leq_abs_inl[\[source\]](#)

```
lemma norm_space_leq_abs_inl : ||v.1.toSpace|| ≤ |v.val.val (Sum.inl 0)|
```

lemma inl_abs_sub_space_norm[\[source\]](#)

```
lemma inl_abs_sub_space_norm :
  0 ≤ |v.val.val (Sum.inl 0)| * |w.val.val (Sum.inl 0)| - ||v.1.toSpace|| * ||w.1.toSpace||
```

def FuturePointing[\[source\]](#)

The future pointing Lorentz vectors with Norm one.

```
def FuturePointing (d : ℕ) : Set (NormOne d)
```

lemma FuturePointing.mem_iff[\[source\]](#)

```
lemma mem_iff : v ∈ FuturePointing d ↔ 0 < v.val.val (Sum.inl 0)
```

lemma mem_iff_inl_nonneg[\[source\]](#)

```
lemma mem_iff_inl_nonneg : v ∈ FuturePointing d ↔ 0 ≤ v.val.val (Sum.inl 0)
```

lemma mem_iff_inl_one_le_inl[\[source\]](#)

```
lemma mem_iff_inl_one_le_inl : v ∈ FuturePointing d ↔ 1 ≤ v.val.val (Sum.inl 0)
```

lemma mem_iff_parity_mem[\[source\]](#)

```
lemma mem_iff_parity_mem : v ∈ FuturePointing d ↔ ((Contr d).p LorentzGroup.parity v.1,
  (NormOne.mem_mulAction _ _).mp v.2) ∈ FuturePointing d
```

lemma not_mem_iff_inl_le_zero[\[source\]](#)

```
lemma not_mem_iff_inl_le_zero : v ∉ FuturePointing d ↔ v.val.val (Sum.inl 0) ≤ 0
```

lemma not_mem_iff_inl_lt_zero[\[source\]](#)

```
lemma not_mem_iff_inl_lt_zero : v ∉ FuturePointing d ↔ v.val.val (Sum.inl 0) < 0
```

lemma not_mem_iff_inl_le_neg_one[\[source\]](#)

```
lemma not_mem_iff_inl_le_neg_one : v ∉ FuturePointing d ↔ v.val.val (Sum.inl 0) ≤ -1
```

lemma not_mem_iff_neg[\[source\]](#)

```
lemma not_mem_iff_neg : v ∉ FuturePointing d ↔ neg v ∈ FuturePointing d
```

lemma inl_nonneg[\[source\]](#)

```
lemma inl_nonneg : 0 ≤ f.val.val.val (Sum.inl 0)
```

lemma abs_inl[\[source\]](#)

```
lemma abs_inl : |f.val.val.val (Sum.inl 0)| = f.val.val.val (Sum.inl 0)
```

lemma inl_eq_sqrt[\[source\]](#)

```
lemma inl_eq_sqrt : f.val.val.val (Sum.inl 0) = √(1 + ||f.1.1.toSpace|| ^ 2)
```

lemma metric_nonneg[\[source\]](#)

```
lemma metric_nonneg : 0 ≤ ⟨⟨f.1.1, f'.1.1⟩⟩m
```

lemma one_add_metric_non_zero[\[source\]](#)

```
lemma one_add_metric_non_zero : 1 + ⟨⟨f.1.1, f'.1.1⟩⟩m ≠ 0
```

lemma metric_reflect_mem_mem[\[source\]](#)

```
lemma metric_reflect_mem_mem (h : v ∈ FuturePointing d) (hw : w ∈ FuturePointing d) :  
  0 ≤ ⟨⟨v.val, (Contr d).p LorentzGroup.parity w.1⟩⟩m
```

lemma metric_reflect_not_mem_not_mem[\[source\]](#)

```
lemma metric_reflect_not_mem_not_mem (h : v ∉ FuturePointing d) (hw : w ∉ FuturePointing d) :  
  0 ≤ ⟨⟨v.val, (Contr d).p LorentzGroup.parity w.1⟩⟩m
```

lemma metric_reflect_mem_not_mem[\[source\]](#)

```
lemma metric_reflect_mem_not_mem (h : v ∈ FuturePointing d) (hw : w ∉ FuturePointing d) :  
  ⟨⟨v.val, (Contr d).p LorentzGroup.parity w.1⟩⟩m ≤ 0
```

lemma metric_reflect_not_mem_mem[\[source\]](#)

```
lemma metric_reflect_not_mem_mem (h : v ∉ FuturePointing d) (hw : w ∈ FuturePointing d) :  
  ⟨⟨v.val, (Contr d).p LorentzGroup.parity w.1⟩⟩m ≤ 0
```

def timeVecNormOneFuture[\[source\]](#)

The FuturePointing d which has all space components zero.

```
@[simps!]  
noncomputable def timeVecNormOneFuture : FuturePointing d
```

def pathFromTime[\[source\]](#)

A continuous path from timeVecNormOneFuture to any other.

```
noncomputable def pathFromTime (u : FuturePointing d) : Path timeVecNormOneFuture u where
```

lemma isPathConnected[\[source\]](#)

```
lemma isPathConnected : IsPathConnected (@Set.univ (FuturePointing d))
```

lemma metric_continuous[\[source\]](#)

```
lemma metric_continuous (u : Contr d) :  
  Continuous (fun (a : FuturePointing d) => ⟨⟨u, a.1.1⟩⟩_m)
```

2.16 HepLean.Lorentz.SL2C.Basic

[HepLean/Lorentz/SL2C/Basic.lean](#) on GitHub.

lemma inverse_coe[\[source\]](#)

```
lemma inverse_coe (M : SL(2, ℂ)) : M.1-1 = (M-1).1
```

lemma transpose_coe[\[source\]](#)

```
lemma transpose_coe (M : SL(2, ℂ)) : M.1T = (M.transpose).1
```

def toSelfAdjointMap[\[source\]](#)

*Given an element $M \in SL(2, \mathbb{C})$ the linear map from selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ) to itself defined by $A \mapsto M * A * M^H$.*

```
@[simps!]  
def toSelfAdjointMap (M : SL(2, ℂ)) :  
  selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ) →ℓ[ℝ] selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ) where
```

lemma toSelfAdjointMap_apply_det[\[source\]](#)

```
lemma toSelfAdjointMap_apply_det (M : SL(2, ℂ)) (A : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)) :  
  det ((toSelfAdjointMap M) A).1 = det A.1
```

lemma toSelfAdjointMap_apply_σSAL_inl[\[source\]](#)

```
lemma toSelfAdjointMap_apply_σSAL_inl (M : SL(2, ℂ)) :  
  toSelfAdjointMap M (PauliMatrix.σSAL (Sum.inl 0)) =  
  ((||M.1 0 0|| ^ 2 + ||M.1 0 1|| ^ 2 + ||M.1 1 0|| ^ 2 + ||M.1 1 1|| ^ 2) / 2) •  
  PauliMatrix.σSAL (Sum.inl 0) +  
  (- ((M.1 0 1).re * (M.1 1 1).re + (M.1 0 1).im * (M.1 1 1).im +  
    (M.1 0 0).im * (M.1 1 0).im + (M.1 0 0).re * (M.1 1 0).re)) • PauliMatrix.σSAL (Sum.inr 0)  
  + ((- (M.1 0 0).re * (M.1 1 0).im + (M.1 1 0).re * (M.1 0 0).im  
    - (M.1 0 1).re * (M.1 1 1).im + (M.1 0 1).im * (M.1 1 1).re)) • PauliMatrix.σSAL (Sum.inr 1)
```

```

+ ((- ||M.1 0 0|| ^ 2 - ||M.1 0 1|| ^ 2 + ||M.1 1 0|| ^ 2 + ||M.1 1 1|| ^ 2) / 2) •
  PauliMatrix.σSAL (Sum.inr 2)

```

def toMatrix[\[source\]](#)

The monoid homomorphisms from $SL(2, \mathbb{C})$ to matrices indexed by $\text{Fin } 1 \oplus \text{Fin } 3$ formed by the action $M \mapsto M^H$.

```

def toMatrix : SL(2, ℂ) →* Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ where

```

lemma toMatrix_apply_contrMod[\[source\]](#)

```

lemma toMatrix_apply_contrMod (M : SL(2, ℂ)) (v : ContrMod 3) :
  (toMatrix M) *v v = ContrMod.toSelfAdjoint.symm
  ((toSelfAdjointMap M) (ContrMod.toSelfAdjoint v))

```

lemma toMatrix_mem_lorentzGroup[\[source\]](#)

```

lemma toMatrix_mem_lorentzGroup (M : SL(2, ℂ)) : toMatrix M ∈ LorentzGroup 3

```

def toLorentzGroup[\[source\]](#)

The group homomorphism from $SL(2, \mathbb{C})$ to the Lorentz group $\mathcal{L}$.

```

@[simps!]
def toLorentzGroup : SL(2, ℂ) →* LorentzGroup 3 where

```

lemma toLorentzGroup_eq_σSAL[\[source\]](#)

```

lemma toLorentzGroup_eq_σSAL (M : SL(2, ℂ)) :
  toLorentzGroup M = LinearMap.toMatrix
  PauliMatrix.σSAL PauliMatrix.σSAL (toSelfAdjointMap M)

```

lemma toSelfAdjointMap_basis[\[source\]](#)

```

lemma toSelfAdjointMap_basis (i : Fin 1 ⊕ Fin 3) :
  toSelfAdjointMap M (PauliMatrix.σSAL i) =
  ∑ j, (toLorentzGroup M).1 j i • PauliMatrix.σSAL j

```

lemma toSelfAdjointMap_σSA[\[source\]](#)

```

lemma toSelfAdjointMap_σSA (i : Fin 1 ⊕ Fin 3) :
  toSelfAdjointMap M (PauliMatrix.σSA i) =
  ∑ j, (toLorentzGroup M-1).1 i j • PauliMatrix.σSA j

```

lemma toLorentzGroup_fst_col[\[source\]](#)

The first column of the Lorentz matrix formed from an element of $SL(2, \mathbb{C})$.

```

lemma toLorentzGroup_fst_col (M : SL(2, ℂ)) :
  (fun μ => (toLorentzGroup M).1 μ (Sum.inl 0)) = fun μ =>
    match μ with
    | Sum.inl 0 => ((||M.1 0 0|| ^ 2 + ||M.1 0 1|| ^ 2 + ||M.1 1 0|| ^ 2 + ||M.1 1 1|| ^ 2) / 2)

```

```

| Sum.inr 0 => (- ((M.1 0 1).re * (M.1 1 1).re + (M.1 0 1).im * (M.1 1 1).im +
  (M.1 0 0).im * (M.1 1 0).im + (M.1 0 0).re * (M.1 1 0).re))
| Sum.inr 1 => ((- (M.1 0 0).re * (M.1 1 0).im + (M.1 1 0).re * (M.1 0 0).im
  - (M.1 0 1).re * (M.1 1 1).im + (M.1 0 1).im * (M.1 1 1).re))
| Sum.inr 2 => ((- ||M.1 0 0|| ^ 2 - ||M.1 0 1|| ^ 2 + ||M.1 1 0|| ^ 2 + ||M.1 1 1|| ^ 2) / 2)

```

lemma toLorentzGroup_inl_inl

[\[source\]](#)

The first element of the image of $SL(2, \mathbb{C})$ in the Lorentz group.

```

lemma toLorentzGroup_inl_inl (M : SL(2, ℂ)) :
  (toLorentzGroup M).1 (Sum.inl 0) (Sum.inl 0) =
  ((||M.1 0 0|| ^ 2 + ||M.1 0 1|| ^ 2 + ||M.1 1 0|| ^ 2 + ||M.1 1 1|| ^ 2) / 2)

```

lemma toLorentzGroup_isOrthochronous

[\[source\]](#)

The image of $SL(2, \mathbb{C})$ in the Lorentz group is orthochronous.

```

lemma toLorentzGroup_isOrthochronous (M : SL(2, ℂ)) :
  LorentzGroup.IsOrthochronous (toLorentzGroup M)

```

informal_lemma toLorentzGroup_det_one

[\[source\]](#)

```

informal_lemma toLorentzGroup_det_one where
  math := "The determinant of the image of `SL(2, ℂ)` in the Lorentz group is one."
  deps := [``toLorentzGroup]

```

informal_lemma toRestrictedLorentzGroup

[\[source\]](#)

```

informal_lemma toRestrictedLorentzGroup where
  math := "The homomorphism from `SL(2, ℂ)` to the restricted Lorentz group."
  deps := [``toLorentzGroup, ``toLorentzGroup_det_one, ``toLorentzGroup_isOrthochronous,
    ``LorentzGroup.Restricted]

```

2.17 HepLean.Mathematics.Fin

[HepLean/Mathematics/Fin.lean](#) on GitHub.

lemma finExtractOnePerm_apply

[\[source\]](#)

```

@[simp]
lemma finExtractOnePerm_apply (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin n.succ.succ)
  (x : Fin n.succ) : finExtractOnePerm i σ x = predAboveI (σ i)
  (σ ((finExtractOne i).symm (Sum.inr x)))

```

lemma finExtractOnePerm_symm_apply

[\[source\]](#)

```

@[simp]
lemma finExtractOnePerm_symm_apply (i : Fin n.succ.succ) (σ : Fin n.succ.succ ≈ Fin n.succ.succ)
  (x : Fin n.succ) : (finExtractOnePerm i σ).symm x = predAboveI (σ.symm (σ i))
  (σ.symm ((finExtractOne (σ i)).symm (Sum.inr x)))

```

def finMapToEquiv[\[source\]](#)

Takes two maps $\text{Fin } n \rightarrow \text{Fin } m$ and returns the equivalence they form.

```
def finMapToEquiv (f1 : Fin n → Fin m) (f2 : Fin m → Fin n)
  (h : ∀ x, f1 (f2 x) = x := by decide)
  (h' : ∀ x, f2 (f1 x) = x := by decide) : Fin n ≈ Fin m where
```

lemma finMapToEquiv_apply[\[source\]](#)

```
@[simp]
lemma finMapToEquiv_apply {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} (x : Fin n) :
  finMapToEquiv f1 f2 h h' x = f1 x
```

lemma finMapToEquiv_symm_apply[\[source\]](#)

```
@[simp]
lemma finMapToEquiv_symm_apply {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} (x : Fin m) :
  (finMapToEquiv f1 f2 h h').symm x = f2 x
```

lemma finMapToEquiv_symm_eq[\[source\]](#)

```
lemma finMapToEquiv_symm_eq {f1 : Fin n → Fin m} {f2 : Fin m → Fin n}
  {h : ∀ x, f1 (f2 x) = x} {h' : ∀ x, f2 (f1 x) = x} :
  (finMapToEquiv f1 f2 h h').symm = finMapToEquiv f2 f1 h' h
```

2.18 HepLean.Meta.Basic

[HepLean/Meta/Basic.lean](#) on GitHub.

def allImports[\[source\]](#)

Gets all imports within HepLean.

```
def allImports : IO (Array Import)
```

def noImports[\[source\]](#)

Number of files within HepLean.

```
def noImports : IO Nat
```

def getConsts[\[source\]](#)

Gets all constants from an import.

```
def Imports.getConsts (imp : Import) : IO (Array ConstantInfo)
```

def getUserConsts[\[source\]](#)

Gets all user defined constants from an import.

```
def Imports.getUserConsts (imp : Import) : MetaM (Array ConstantInfo)
```

def getLines[\[source\]](#)*Lines from import.*

```
def Imports.getLines (imp : Import) : IO (Array String)
```

def toFile[\[source\]](#)*Turns a name into a Lean file.*

```
def Name.toFile (c : Name) : MetaM String
```

def lineNumber[\[source\]](#)*Given a name, returns the line number.*

```
def Name.lineNumber (c : Name) : MetaM Nat
```

def location[\[source\]](#)*Returns the location of a name.*

```
def Name.location (c : Name) : MetaM String
```

def hasPos[\[source\]](#)*Determines if a name has a location.*

```
def Name.hasPos (c : Name) : MetaM Bool
```

def hasDocString[\[source\]](#)*Determines if a name has a doc string.*

```
def Name.hasDocString (c : Name) : MetaM Bool
```

def noDefs[\[source\]](#)*Number of definitions.*

```
def noDefs : MetaM Nat
```

def noLemmas[\[source\]](#)*Number of definitions.*

```
def noLemmas : MetaM Nat
```

def noDefsNoDocString [\[source\]](#)

Number of definitions without a doc-string.

```
def noDefsNoDocString : MetaM Nat
```

def noLemmasNoDocString [\[source\]](#)

Number of definitions without a doc-string.

```
def noLemmasNoDocString : MetaM Nat
```

def noLines [\[source\]](#)

The number of lines in HepLean.

```
def noLines : IO Nat
```

def noInformalLemmas [\[source\]](#)

The number of informal lemmas in HepLean.

```
def noInformalLemmas : MetaM Nat
```

def noInformalDefs [\[source\]](#)

The number of informal definitions in HepLean.

```
def noInformalDefs : MetaM Nat
```

def noTODOs [\[source\]](#)

The number of TODO items.

```
def noTODOs : IO Nat
```

def noFilesWithTODOs [\[source\]](#)

The number of files with a TODO item.

```
def noFilesWithTODOs : IO Nat
```

2.19 HepLean.Tensors.OverColor.Basic

[HepLean/Tensors/OverColor/Basic.lean](#) on GitHub.

lemma hom_comp [\[source\]](#)

@[simp]

```
lemma hom_comp (m : f → g) (n : g → h) : (m >> n).hom = m.hom >> n.hom
```

lemma mk_left[\[source\]](#)

```
@[simp]
lemma mk_left (f : X → C) : (mk f).left = X
```

lemma β _hom_toEquiv[\[source\]](#)

```
@[simp]
lemma  $\beta$ _hom_toEquiv (f : X → C) (g : Y → C) :
  Hom.toEquiv ( $\beta$ _ (OverColor.mk f) (OverColor.mk g)).hom = Equiv.sumComm X Y
```

lemma β _inv_toEquiv[\[source\]](#)

```
@[simp]
lemma  $\beta$ _inv_toEquiv (f : X → C) (g : Y → C) :
  Hom.toEquiv ( $\beta$ _ (OverColor.mk f) (OverColor.mk g)).inv = Equiv.sumComm Y X
```

lemma whiskeringLeft_toEquiv[\[source\]](#)

```
@[simp]
lemma whiskeringLeft_toEquiv (f : X → C) (g : Y → C) (h : Z → C)
  ( $\sigma$  : OverColor.mk f → OverColor.mk g) :
  Hom.toEquiv (OverColor.mk h  $\triangleleft$   $\sigma$ ) = (Equiv.refl Z).sumCongr (Hom.toEquiv  $\sigma$ )
```

lemma whiskeringRight_toEquiv[\[source\]](#)

```
@[simp]
lemma whiskeringRight_toEquiv (f : X → C) (g : Y → C) (h : Z → C)
  ( $\sigma$  : OverColor.mk f → OverColor.mk g) :
  Hom.toEquiv ( $\sigma$   $\triangleright$  OverColor.mk h) = (Hom.toEquiv  $\sigma$ ).sumCongr (Equiv.refl Z)
```

lemma α _hom_toEquiv[\[source\]](#)

```
@[simp]
lemma  $\alpha$ _hom_toEquiv (f : X → C) (g : Y → C) (h : Z → C) :
  Hom.toEquiv ( $\alpha$ _ (OverColor.mk f) (OverColor.mk g) (OverColor.mk h)).hom =
  Equiv.sumAssoc X Y Z
```

lemma α _inv_toEquiv[\[source\]](#)

```
@[simp]
lemma  $\alpha$ _inv_toEquiv (f : X → C) (g : Y → C) (h : Z → C) :
  Hom.toEquiv ( $\alpha$ _ (OverColor.mk f) (OverColor.mk g) (OverColor.mk h)).inv =
  (Equiv.sumAssoc X Y Z).symm
```

2.20 HepLean.Tensors.OverColor.Discrete

[HepLean/Tensors/OverColor/Discrete.lean](#) on GitHub.

lemma pairIsoSep_inv_tprod[\[source\]](#)

```
lemma pairIsoSep_inv_tprod {c1 c2 : C} (fx : (i : (1 Type).obj (OverColor.mk ![c1, c2]).left) →
  CoeSort.coe (F.obj { as := (OverColor.mk ![c1, c2]).hom i }))) :
```

```
(pairIsoSep F).inv.hom (PiTensorProduct.tprod k fx) = fx (0 : Fin 2) ⊗t fx (1 : Fin 2)
```

lemma pairIsoSep_β_perm_cond

[\[source\]](#)

```
lemma pairIsoSep_β_perm_cond (c1 c2 : C) : ∀ (x : Fin (Nat.succ 0).succ), ![c2, c1] x =
  (![c1, c2] ∘ ↑(finMapToEquiv ![1, 0] ![1, 0])).symm x
```

lemma pairIsoSep_β

[\[source\]](#)

```
lemma pairIsoSep_β {c1 c2 : C} (x : ↑(F.obj { as := c1 } ⊗ F.obj { as := c2 })).V :
  (Discrete.pairIsoSep F).hom.hom ((β_ (F.obj (Discrete.mk c1)) _).hom.hom x) =
  ((lift.obj F).map ((OverColor.equivToHomEq (finMapToEquiv ![1, 0] ![1, 0])
    (pairIsoSep_β_perm_cond c1 c2))))).hom
  ((Discrete.pairIsoSep F).hom.hom x)
```

lemma rep_iso_inv_hom_apply

[\[source\]](#)

@[simp]

```
lemma rep_iso_inv_hom_apply (x y : Rep k G) (f : x ≅ y) (i : x) :
  f.inv.hom (f.hom.hom i) = i
```

lemma rep_iso_hom_inv_apply

[\[source\]](#)

@[simp]

```
lemma rep_iso_hom_inv_apply (x y : Rep k G) (f : x ≅ y) (i : y) :
  f.hom.hom (f.inv.hom i) = i
```

lemma rep_iso_apply_iff

[\[source\]](#)

```
lemma rep_iso_apply_iff (x y : Rep k G) (f : x ≅ y) (i : x) (j : y) :
  f.hom.hom i = j ↔ i = f.inv.hom j
```

lemma rep_iso_inv_apply_iff

[\[source\]](#)

```
lemma rep_iso_inv_apply_iff (x y : Rep k G) (f : x ≅ y) (i : y) (j : x) :
  f.inv.hom i = j ↔ i = f.hom.hom j
```

2.21 HepLean.Tensors.OverColor.Iso

[HepLean/Tensors/OverColor/Iso.lean](#) on GitHub.

lemma equivToIso_inv_homToEquiv

[\[source\]](#)

@[simp]

```
lemma equivToIso_inv_homToEquiv {c : X → C} (e : X ≅ Y) :
  Hom.toEquiv (equivToIso (c := c) e).inv = e.symm
```

lemma mkIso_hom_hom_left

[\[source\]](#)

```
lemma mkIso_hom_hom_left {c1 c2 : X → C} (h : c1 = c2) : (mkIso h).hom.hom.left =
  (Equiv.refl X).toFun
```

lemma mkIso_hom_hom_left_apply[\[source\]](#)

```
@[simp]
lemma mkIso_hom_hom_left_apply {c1 c2 : X → C} (h : c1 = c2) (x : X) :
  (mkIso h).hom.hom.left x = x
```

lemma equivToHomEq_hom_left[\[source\]](#)

```
@[simp]
lemma equivToHomEq_hom_left {c : X → C} {c1 : Y → C} (e : X ≈ Y)
  (h : ∀ x, c1 x = (c ∘ e.symm) x := by decide) : (equivToHomEq e h).hom.left =
  e.toFun
```

lemma equivToHomEq_toEquiv[\[source\]](#)

```
@[simp]
lemma equivToHomEq_toEquiv {c : X → C} {c1 : Y → C} (e : X ≈ Y)
  (h : ∀ x, c1 x = (c ∘ e.symm) x := by decide) : Hom.toEquiv (equivToHomEq e h) = e
```

lemma extractTwo_hom_left_apply[\[source\]](#)

```
@[simp]
lemma extractTwo_hom_left_apply {n : ℕ} (i : Fin n.succ.succ.succ) (j : Fin n.succ.succ)
  {c1 c2 : Fin n.succ.succ.succ → C} (σ : mk c1 → mk c2) (x : Fin n.succ) :
  (extractTwo i j σ).hom.left x =
    (finExtractOnePerm ((finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j)
      (finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ))) x
```

lemma extractTwo_toEquiv[\[source\]](#)

```
@[simp]
lemma extractTwo_toEquiv {n : ℕ} (i : Fin n.succ.succ.succ) (j : Fin n.succ.succ)
  {c1 c2 : Fin n.succ.succ.succ → C} (σ : mk c1 → mk c2) :
  (Hom.toEquiv (extractTwo i j σ)) =
    (finExtractOnePerm ((finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j)
      (finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)))
```

lemma extractTwo_toEquiv_symm[\[source\]](#)

```
lemma extractTwo_toEquiv_symm {n : ℕ} (i : Fin n.succ.succ.succ) (j : Fin n.succ.succ)
  {c1 c2 : Fin n.succ.succ.succ → C} (σ : mk c1 → mk c2) :
  (Hom.toEquiv (extractTwo i j σ)).symm =
    (finExtractOnePerm ((finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ)).symm j)
      (finExtractOnePerm ((Hom.toEquiv σ).symm i) (Hom.toEquiv σ))).symm
```

2.22 HepLean.Tensors.OverColor.Lift

[HepLean/Tensors/OverColor/Lift.lean](#) on GitHub.

lemma forgetLiftApp_naturality_eqToHom[\[source\]](#)

```
lemma forgetLiftApp_naturality_eqToHom (c c1 : C) (h : c = c1) :
  (forgetLiftApp F c).hom >> F.map (Discrete.eqToHom h) =
  (lift.obj F).map (OverColor.mkIso (by rw [h])).hom >> (forgetLiftApp F c1).hom
```

lemma forgetLiftApp_naturality_eqToHom_apply[\[source\]](#)

```
lemma forgetLiftApp_naturality_eqToHom_apply (c c1 : C) (h : c = c1)
  (x : (lift.obj F).obj (OverColor.mk (fun _ : Fin 1 => c))) :
  (F.map (Discrete.eqToHom h)).hom ((forgetLiftApp F c).hom.hom x) =
  (forgetLiftApp F c1).hom.hom (((lift.obj F).map (OverColor.mkIso (by rw [h]))).hom).hom x)
```

def forgetLiftAppCon[\[source\]](#)

The isomorphism between the representation $(\text{lift.obj } F).obj (\text{OverColor.mk } ![c])$ and the representation $F.obj (\text{Discrete.mk } c)$. See `forgetLiftApp` for an alternative version.

```
def forgetLiftAppCon (c : C) : (lift.obj F).obj (OverColor.mk ![c])
  ≡ F.obj (Discrete.mk c)
```

lemma forgetLiftAppCon_inv_apply_expand[\[source\]](#)

```
lemma forgetLiftAppCon_inv_apply_expand (c : C) (x : F.obj (Discrete.mk c)) :
  (forgetLiftAppCon F c).inv.hom x =
  ((lift.obj F).map (OverColor.mkIso (by
  funext i
  fin_cases i
  rfl))).hom).hom ((forgetLiftApp F c).inv.hom x)
```

lemma forgetLiftAppCon_naturality_eqToHom[\[source\]](#)

```
lemma forgetLiftAppCon_naturality_eqToHom (c c1 : C) (h : c = c1) :
  (forgetLiftAppCon F c).hom >> F.map (Discrete.eqToHom h) =
  (lift.obj F).map (OverColor.mkIso (by rw [h])).hom >> (forgetLiftAppCon F c1).hom
```

lemma forgetLiftAppCon_naturality_eqToHom_apply[\[source\]](#)

```
lemma forgetLiftAppCon_naturality_eqToHom_apply (c c1 : C) (h : c = c1)
  (x : (lift.obj F).obj (OverColor.mk ![c])) :
  (F.map (Discrete.eqToHom h)).hom ((forgetLiftAppCon F c).hom.hom x) =
  (forgetLiftAppCon F c1).hom.hom
  (((lift.obj F).map (OverColor.mkIso (by rw [h]))).hom).hom x)
```

2.23 HepLean.Tensors.TensorSpecies.Basic

[HepLean/Tensors/TensorSpecies/Basic.lean](#) on GitHub.

def tensorToVec[\[source\]](#)

The equivalence between tensors based on $![c]$ and vectors in $S.FD.obj (\text{Discrete.mk } c)$.

```
def tensorToVec (c : S.C) : S.F.obj (OverColor.mk ![c]) ≡ S.FD.obj (Discrete.mk c)
```

lemma tensorToVec_inv_apply_expand[\[source\]](#)

```
lemma tensorToVec_inv_apply_expand (c : S.C) (x : S.FD.obj (Discrete.mk c)) :
  (S.tensorToVec c).inv.hom x =
```

```
((lift.obj S.FD).map (OverColor.mkIso (by
funext i
fin_cases i
rfl)).hom).hom ((OverColor.forgetLiftApp S.FD c).inv.hom x)
```

lemma tensorToVec_naturality_eqToHom

[\[source\]](#)

```
lemma tensorToVec_naturality_eqToHom (c c1 : S.C) (h : c = c1) :
(S.tensorToVec c).hom >> S.FD.map (Discrete.eqToHom h) =
S.F.map (OverColor.mkIso (by rw [h])).hom >> (S.tensorToVec c1).hom
```

lemma tensorToVec_naturality_eqToHom_apply

[\[source\]](#)

```
lemma tensorToVec_naturality_eqToHom_apply (c c1 : S.C) (h : c = c1)
(x : S.F.obj (OverColor.mk ![c])) :
(S.FD.map (Discrete.eqToHom h)).hom ((S.tensorToVec c).hom.hom x) =
(S.tensorToVec c1).hom.hom (((S.F.map (OverColor.mkIso (by rw [h])).hom).hom x))
```

2.24 HepLean.Tensors.TensorSpecies.Contractions.Basic

[HepLean/Tensors/TensorSpecies/Contractions/Basic.lean](#) on GitHub.

def contractSelfHom

[\[source\]](#)

The equivariant map from $S.FD.obj (Discrete.mk c) \otimes S.FD.obj (Discrete.mk c)$ to the underlying field obtained by contracting.

```
def contractSelfHom (c : S.C) : S.FD.obj (Discrete.mk c) \otimes S.FD.obj (Discrete.mk c) \longrightarrow
\mathbb{K}_\text{ } (Rep S.k S.G)
```

def contractSelfField

[\[source\]](#)

The contraction of two vectors in a tensor species of the same color, as a linear map to the underlying field.

```
def contractSelfField {S : TensorSpecies} {c : S.C} :
S.FD.obj (Discrete.mk c) \otimes [S.k] S.FD.obj (Discrete.mk c) \rightarrow_l [S.k] S.k
```

lemma contractSelfField_equivariant

[\[source\]](#)

The map contractSelfField is equivariant with respect to the group action.

```
@[simp]
lemma contractSelfField_equivariant {S : TensorSpecies} {c : S.C} {g : S.G}
(\psi : S.FD.obj (Discrete.mk c)) (\phi : S.FD.obj (Discrete.mk c)) :
\langle\langle S.FD.obj (Discrete.mk c) \rangle\rangle.\rho g \psi, (S.FD.obj (Discrete.mk c) \rangle\rangle.\rho g \phi \rangle\rangle_{t_S} = \langle\langle \psi, \phi \rangle\rangle_{t_S}
```

informal_lemma contractSelfField_non_degenerate

[\[source\]](#)

```
informal_lemma contractSelfField_non_degenerate where
math := "The contraction of two vectors of the same color is non-degenerate.
I.e.  $\langle\langle \psi, \phi \rangle\rangle_{t_S} = 0$  for all  $\phi$  implies  $\psi = 0$ ."
proof := "The basic idea is that being degenerate contradicts the assumption of having a unit
in the tensor species."
```

```
deps := [``contractSelfField]
```

informal_lemma contractSelfField_tensorTree

[\[source\]](#)

```
informal_lemma contractSelfField_tensorTree where
  math := "The contraction  $\langle\langle\psi, \phi\rangle\rangle_{\tau_S}$  is related to the tensor tree
     $\{\psi \mid \mu \otimes (S.dualRepIsoDiscrete\ c).hom\ \phi \mid \mu\}^T$  "
  deps := [``contractSelfField, ``TensorTree]
```

def IsNormOne

[\[source\]](#)

A vector satisfies IsNormOne if it has norm equal to one, i.e. if $\langle\langle\psi, \psi\rangle\rangle_{\tau_S} = 1$.

```
def IsNormOne {c : S.C} (ψ : S.FD.obj (Discrete.mk c)) : Prop
```

lemma action_isNormOne_of_isNormOne

[\[source\]](#)

If a vector is norm-one, then any vector in the orbit of that vector is also norm-one.

```
@[simp]
lemma action_isNormOne_of_isNormOne {c : S.C} {ψ : S.FD.obj (Discrete.mk c)} (g : S.G) :
  S.IsNormOne ((S.FD.obj (Discrete.mk c)).ρ g ψ) ↔ S.IsNormOne ψ
```

def IsNormZero

[\[source\]](#)

A vector satisfies IsNormZero if it has norm equal to zero, i.e. if $\langle\langle\psi, \psi\rangle\rangle_{\tau_S} = 0$.

```
def IsNormZero {c : S.C} (ψ : S.FD.obj (Discrete.mk c)) : Prop
```

lemma zero_isNormZero

[\[source\]](#)

The zero vector has norm equal to zero.

```
@[simp]
lemma zero_isNormZero {c : S.C} : @IsNormZero S c 0
```

lemma smul_isNormZero_of_isNormZero

[\[source\]](#)

If a vector is norm-zero, then any scalar multiple of that vector is also norm-zero.

```
lemma smul_isNormZero_of_isNormZero {c : S.C} {ψ : S.FD.obj (Discrete.mk c)}
  (h : S.IsNormZero ψ) (a : S.k) : S.IsNormZero (a • ψ)
```

lemma action_isNormZero_iff_isNormZero

[\[source\]](#)

If a vector is norm-zero, then any vector in the orbit of that vector is also norm-zero.

```
@[simp]
lemma action_isNormZero_iff_isNormZero {c : S.C} {ψ : S.FD.obj (Discrete.mk c)} (g : S.G) :
  S.IsNormZero ((S.FD.obj (Discrete.mk c)).ρ g ψ) ↔ S.IsNormZero ψ
```

2.25 HepLean.Tensors.TensorSpecies.Contractions.Categorical

[HepLean/Tensors/TensorSpecies/Contractions/Categorical.lean](#) on GitHub.

def contrOneTwoLeft[\[source\]](#)

Th map built contracting a 1-tensor with a 2-tensor using basic categorical consstruc-tions.s

```
def contrOneTwoLeft {c1 c2 : S.C}
  (x : S.F.obj (OverColor.mk ![c1])) (y : S.F.obj (OverColor.mk ![S.τ c1, c2])) :
  S.F.obj (OverColor.mk ![c2])
```

lemma contrOneTwoLeft_smul_left[\[source\]](#)

```
@[simp]
lemma contrOneTwoLeft_smul_left {c1 c2 : S.C} (x : S.F.obj (OverColor.mk ![c1]))
  (y : S.F.obj (OverColor.mk ![S.τ c1, c2])) (r : S.k) :
  contrOneTwoLeft (r • x) y = r • contrOneTwoLeft x y
```

lemma contrOneTwoLeft_smul_right[\[source\]](#)

```
@[simp]
lemma contrOneTwoLeft_smul_right {c1 c2 : S.C} (x : S.F.obj (OverColor.mk ![c1]))
  (y : S.F.obj (OverColor.mk ![S.τ c1, c2])) (r : S.k) :
  contrOneTwoLeft x (r • y) = r • contrOneTwoLeft x y
```

lemma contrOneTwoLeft_add_left[\[source\]](#)

```
@[simp]
lemma contrOneTwoLeft_add_left {c1 c2 : S.C} (x y : S.F.obj (OverColor.mk ![c1]))
  (z : S.F.obj (OverColor.mk ![S.τ c1, c2])) :
  contrOneTwoLeft (x + y) z = contrOneTwoLeft x z + contrOneTwoLeft y z
```

lemma contrOneTwoLeft_add_right[\[source\]](#)

```
@[simp]
lemma contrOneTwoLeft_add_right {c1 c2 : S.C} (x : S.F.obj (OverColor.mk ![c1]))
  (y z : S.F.obj (OverColor.mk ![S.τ c1, c2])) :
  contrOneTwoLeft x (y + z) = contrOneTwoLeft x y + contrOneTwoLeft x z
```

lemma contrOneTwoLeft_tprod_eq[\[source\]](#)

```
lemma contrOneTwoLeft_tprod_eq {c1 c2 : S.C}
  (fx : (i : (1 Type).obj (OverColor.mk ![c1])).left) →
  CoeSort.coe (S.FD.obj { as := (OverColor.mk ![c1]).hom i }))
  (fy : (i : (1 Type).obj (OverColor.mk ![S.τ c1, c2])).left)
  → CoeSort.coe (S.FD.obj { as := (OverColor.mk ![S.τ c1, c2]).hom i })) :
  contrOneTwoLeft (PiTensorProduct.tprod S.k fx) (PiTensorProduct.tprod S.k fy) =
  ((S.tensorToVec c2).inv.hom
    (((S.contr.app (Discrete.mk c1)).hom (fx (0 : Fin 1) ⊗ fy (0 : Fin 2)) •
      fy (1 : Fin 2))))
```

lemma contr_one_two_left_eq_contrOneTwoLeft_tprod[\[source\]](#)

```
lemma contr_one_two_left_eq_contrOneTwoLeft_tprod {c1 c2 : S.C} (x : S.F.obj (OverColor.mk ![c1]))
  (y : S.F.obj (OverColor.mk ![S.τ c1, c2]))
```

```

(fx : (i : (1 Type).obj (OverColor.mk ![c1]).left) →
  CoeSort.coe (S.FD.obj { as := (OverColor.mk ![c1]).hom i }))
(fy : (i : (1 Type).obj (OverColor.mk ![S.τ c1, c2]).left)
  → CoeSort.coe (S.FD.obj { as := (OverColor.mk ![S.τ c1, c2]).hom i }))
(hx : x = PiTensorProduct.tprod S.k fx)
(hy : y = PiTensorProduct.tprod S.k fy) :
{x | μ ⊗ y | μ v}^T.tensor =
(S.F.mapIso (OverColor.mkIso (by funext x; fin_cases x; rfl))).hom.hom
(contrOneTwoLeft x y)

```

lemma contr_one_two_left_eq_contrOneTwoLeft

[\[source\]](#)

```

lemma contr_one_two_left_eq_contrOneTwoLeft {c1 c2 : S.C} (x : S.F.obj (OverColor.mk ![c1]))
(y : S.F.obj (OverColor.mk ![S.τ c1, c2])) :
{x | μ ⊗ y | μ v}^T.tensor = (S.F.map (OverColor.mkIso (by funext x; fin_cases x; rfl))).hom
.hom
(contrOneTwoLeft x y)

```

lemma contrOneTwoLeft_tensorTree

[\[source\]](#)

Expanding contrOneTwoLeft as a tensor tree.

```

lemma contrOneTwoLeft_tensorTree {c1 c2 : S.C} (x : S.F.obj (OverColor.mk ![c1]))
(y : S.F.obj (OverColor.mk ![S.τ c1, c2])) :
(contrOneTwoLeft x y) = ({x | μ ⊗ y | μ v}^T |>
  perm (OverColor.equivToHomEq (Equiv.refl _) (fun x => by fin_cases x; rfl))).tensor

```

lemma contr_two_two_inner_tprod

[\[source\]](#)

Expands the inner contraction of two 2-tensors which are tprods in terms of basic categorical constructions and fields of the tensor species.

```

lemma contr_two_two_inner_tprod (c : S.C) (x : S.F.obj (OverColor.mk ![c, c]))
(fx : (i : (1 Type).obj (OverColor.mk ![c, c]).left) →
  CoeSort.coe (S.FD.obj { as := (OverColor.mk ![c, c]).hom i }))
(y : S.F.obj (OverColor.mk ![S.τ c], (S.τ c)))
(fy : (i : (1 Type).obj (OverColor.mk ![S.τ c, S.τ c]).left) →
  CoeSort.coe (S.FD.obj { as := (OverColor.mk ![S.τ c, S.τ c]).hom i }))
(hx : x = PiTensorProduct.tprod S.k fx)
(hy : y = PiTensorProduct.tprod S.k fy) :
{x | μ v ⊗ y | v p}^T.tensor = (S.F.map (OverColor.mkIso (by
  funext x
    fin_cases x <=> rfl)).hom).hom ((OverColor.Discrete.pairIsoSep S.FD).hom.hom
  (((S.FD.obj (Discrete.mk c)) < (λ_ (S.FD.obj (Discrete.mk (S.τ c)))).hom).hom
  (((S.FD.obj (Discrete.mk c)) < ((S.contr.app (Discrete.mk c)) >
  (S.FD.obj (Discrete.mk (S.τ c)))).hom
  (((S.FD.obj (Discrete.mk c)) < (α_ (S.FD.obj (Discrete.mk (c)))
    (S.FD.obj (Discrete.mk (S.τ c))) (S.FD.obj (Discrete.mk (S.τ c)))).inv).hom
  ((α_ (S.FD.obj (Discrete.mk (c))) (S.FD.obj (Discrete.mk (c)))
    (S.FD.obj (Discrete.mk (S.τ c)) ⊗ S.FD.obj (Discrete.mk (S.τ c)))).hom.hom
  ((OverColor.Discrete.pairIsoSep S.FD).inv.hom x ⊗_t
  (OverColor.Discrete.pairIsoSep S.FD).inv.hom y))))))

```

lemma contr_two_two_inner

[\[source\]](#)

Expands the inner contraction of two 2-tensors in terms of basic categorical constructions and fields of the tensor species.

```
lemma contr_two_two_inner (c : S.C) (x : S.F.obj (OverColor.mk ![c, c]))
  (y : S.F.obj (OverColor.mk ![(S.τ c), (S.τ c)])) :
  {x | μ v ⊗ y | v p}^T.tensor = (S.F.map (OverColor.mkIso (by
    funext x
      fin_cases x <=> rfl)).hom).hom ((OverColor.Discrete.pairIsoSep S.FD).hom.hom
    (((S.FD.obj (Discrete.mk c)) < (λ_ (S.FD.obj (Discrete.mk (S.τ c)))).hom).hom
    (((S.FD.obj (Discrete.mk c)) < ((S.contr.app (Discrete.mk c)) >
    (S.FD.obj (Discrete.mk (S.τ c)))).hom
    (((S.FD.obj (Discrete.mk c)) < (α_ (S.FD.obj (Discrete.mk (c)))
    (S.FD.obj (Discrete.mk (S.τ c))) (S.FD.obj (Discrete.mk (S.τ c))).inv).hom
    ((α_ (S.FD.obj (Discrete.mk (c))) (S.FD.obj (Discrete.mk (c)))
    (S.FD.obj (Discrete.mk (S.τ c)) ⊗ S.FD.obj (Discrete.mk (S.τ c))).hom.hom
    ((OverColor.Discrete.pairIsoSep S.FD).inv.hom x ⊗_t
    (OverColor.Discrete.pairIsoSep S.FD).inv.hom y)))))))
```

2.26 HepLean.Tensors.TensorSpecies.DualRepIso

[HepLean/Tensors/TensorSpecies/DualRepIso.lean](#) on GitHub.

def toDualRep

[\[source\]](#)

The morphism from $S.FD.obj (Discrete.mk c)$ to $S.FD.obj (Discrete.mk (S.τ c))$ defined by contracting with the metric.

```
def toDualRep (c : S.C) : S.FD.obj (Discrete.mk c) → S.FD.obj (Discrete.mk (S.τ c))
```

lemma toDualRep_congr

[\[source\]](#)

The toDualRep for equal colors is the same, up-to conjugation by a trivial equivalence.

```
lemma toDualRep_congr {c c' : S.C} (h : c = c') : S.toDualRep c = S.FD.map (Discrete.eqToHom h)
  >>
  S.toDualRep c' >> S.FD.map (Discrete.eqToHom (congrArg S.τ h.symm))
```

def fromDualRep

[\[source\]](#)

The morphism from $S.FD.obj (Discrete.mk (S.τ c))$ to $S.FD.obj (Discrete.mk c)$ defined by contracting with the metric.

```
def fromDualRep (c : S.C) : S.FD.obj (Discrete.mk (S.τ c)) → S.FD.obj (Discrete.mk c)
```

lemma toDualRep_apply_eq_contrOneTwoLeft

[\[source\]](#)

The rewriting of toDualRep in terms of contrOneTwoLeft.

```
lemma toDualRep_apply_eq_contrOneTwoLeft (c : S.C) (x : S.FD.obj (Discrete.mk c)) :
  (S.toDualRep c).hom x = (S.tensorToVec (S.τ c)).hom.hom
  (contrOneTwoLeft (((S.tensorToVec c).inv.hom x)
  (S.metricTensor (S.τ c)))
```

lemma toDualRep_tensorTree[\[source\]](#)

Expansion of toDualRep is $(S.\text{tensorToVec } c).\text{inv.hom } x \mid \mu \otimes S.\text{metricTensor } (S.\tau c) \mid \mu v$.

```
lemma toDualRep_tensorTree (c : S.C) (x : S.FD.obj (Discrete.mk c)) :
  let y : S.F.obj (OverColor.mk ![c])
```

lemma fromDualRep_tensorTree[\[source\]](#)

```
lemma fromDualRep_tensorTree (c : S.C) (x : S.FD.obj (Discrete.mk (S.τ c))) :
  let y : S.F.obj (OverColor.mk ![S.τ c])
```

lemma toDualRep_fromDualRep_tensorTree_metrics[\[source\]](#)

Applying toDualRep followed by fromDualRep is equivalent to contracting with two metric tensors on the right.

```
lemma toDualRep_fromDualRep_tensorTree_metrics (c : S.C) (x : S.FD.obj (Discrete.mk c)) :
  let y : S.F.obj (OverColor.mk ![c])
```

lemma toDualRep_fromDualRep_tensorTree_unitTensor[\[source\]](#)

Applying toDualRep followed by fromDualRep is equivalent to contracting with a unit tensors on the right.

```
lemma toDualRep_fromDualRep_tensorTree_unitTensor (c : S.C) (x : S.FD.obj (Discrete.mk c)) :
  let y : S.F.obj (OverColor.mk ![c])
```

lemma toDualRep_fromDualRep_tensorTree[\[source\]](#)

```
lemma toDualRep_fromDualRep_tensorTree (c : S.C) (x : S.FD.obj (Discrete.mk c)) :
  let y : S.F.obj (OverColor.mk ![c])
```

lemma toDualRep_fromDualRep_eq_self[\[source\]](#)

```
lemma toDualRep_fromDualRep_eq_self (c : S.C) (x : S.FD.obj (Discrete.mk c)) :
  (S.fromDualRep c).hom ((S.toDualRep c).hom x) = x
```

lemma fromDualRep_toDualRep_eq_self[\[source\]](#)

```
lemma fromDualRep_toDualRep_eq_self (c : S.C) (x : S.FD.obj (Discrete.mk (S.τ c))) :
  (S.toDualRep c).hom ((S.fromDualRep c).hom x) = x
```

def dualRepIsoDiscrete[\[source\]](#)

The isomorphism between the representation associated with a color, and that associated with its dual.

```
def dualRepIsoDiscrete (c : S.C) : S.FD.obj (Discrete.mk c) ≅ S.FD.obj (Discrete.mk (S.τ c))
  where
```

informal_definition dualRepIso[\[source\]](#)

```
informal_definition dualRepIso where
  math := "Given a `i : Fin n` the isomorphism between `S.F.obj (OverColor.mk c)` and
    `S.F.obj (OverColor.mk (Function.update c i (S.τ (c i))))` induced by `dualRepIsoDiscrete`
    acting on the `i`-th component of the color."
  deps := [``dualRepIsoDiscrete]
```

informal_lemma dualRepIso_unitTensor_fst[\[source\]](#)

```
informal_lemma dualRepIso_unitTensor_fst where
  math := "Acting with `dualRepIso` on the fst component of a `unitTensor` returns a metric."
  deps := [``dualRepIso, ``unitTensor, ``metricTensor]
```

informal_lemma dualRepIso_unitTensor_snd[\[source\]](#)

```
informal_lemma dualRepIso_unitTensor_snd where
  math := "Acting with `dualRepIso` on the snd component of a `unitTensor` returns a metric."
  deps := [``dualRepIso, ``unitTensor, ``metricTensor]
```

informal_lemma dualRepIso_metricTensor_fst[\[source\]](#)

```
informal_lemma dualRepIso_metricTensor_fst where
  math := "Acting with `dualRepIso` on the fst component of a `metricTensor` returns a
    unitTensor."
  deps := [``dualRepIso, ``unitTensor, ``metricTensor]
```

informal_lemma dualRepIso_metricTensor_snd[\[source\]](#)

```
informal_lemma dualRepIso_metricTensor_snd where
  math := "Acting with `dualRepIso` on the snd component of a `metricTensor` returns a
    unitTensor."
  deps := [``dualRepIso, ``unitTensor, ``metricTensor]
```

2.27 HepLean.Tensors.TensorSpecies.MetricTensor

[HepLean/Tensors/TensorSpecies/MetricTensor.lean](#) on GitHub.

def metricTensor[\[source\]](#)

The metric of a tensor species in a PiTensorProduct.

```
def metricTensor (S : TensorSpecies) (c : S.C) : S.F.obj (OverColor.mk ![c, c])
```

lemma metricTensor_congr[\[source\]](#)

```
lemma metricTensor_congr {c c' : S.C} (h : c = c') : {S.metricTensor c | μ v}^T.tensor =
  (perm (OverColor.equivToHomEq (Equiv.refl _) (fun x => by subst h; fin_cases x <=> rfl))
    {S.metricTensor c' | μ v}^T).tensor
```

lemma pairIsoSep_inv_metricTensor[\[source\]](#)

```

lemma pairIsoSep_inv_metricTensor (c : S.C) :
  (Discrete.pairIsoSep S.FD).inv.hom (S.metricTensor c) =
  (S.metric.app (Discrete.mk c)).hom (1 : S.k)

```

lemma `contr_metric_braid_unit`

[\[source\]](#)

Contraction of a metric tensor with a metric tensor gives the unit. Like `S.contr_metric` but with the braiding appearing on the side of the unit.

```

lemma contr_metric_braid_unit (c : S.C) : (((S.FD.obj (Discrete.mk c)) <
  (λ_ (S.FD.obj (Discrete.mk (S.τ c))))).hom).hom
  (((S.FD.obj (Discrete.mk c)) < ((S.contr.app (Discrete.mk c)) ▷
  (S.FD.obj (Discrete.mk (S.τ c))))).hom
  (((S.FD.obj (Discrete.mk c)) < (α_ (S.FD.obj (Discrete.mk (c)))
  (S.FD.obj (Discrete.mk (S.τ c)))) (S.FD.obj (Discrete.mk (S.τ c))))).inv).hom
  ((α_ (S.FD.obj (Discrete.mk (c))) (S.FD.obj (Discrete.mk (c)))
  (S.FD.obj (Discrete.mk (S.τ c))) ⊗ S.FD.obj (Discrete.mk (S.τ c))))).hom.hom
  (((OverColor.Discrete.pairIsoSep S.FD).inv.hom (S.metricTensor c) ⊗τ
  (OverColor.Discrete.pairIsoSep S.FD).inv.hom (S.metricTensor (S.τ c)))))) =
  (β_ (S.FD.obj (Discrete.mk (S.τ c))) (S.FD.obj (Discrete.mk c))).hom.hom
  ((S.unit.app (Discrete.mk c)).hom (1 : S.k))

```

lemma `metricTensor_contr_dual_metricTensor_perm_cond`

[\[source\]](#)

```

lemma metricTensor_contr_dual_metricTensor_perm_cond (c : S.C) : ∀ (x : Fin (Nat.succ 0).succ),
  ((Sum.elim ![c, c] ![S.τ c, S.τ c] ∘ †finSumFinEquiv.symm) ∘
  Fin.succAbove 1 ∘ Fin.succAbove 1) x =
  (![S.τ c, c] ∘ †(finMapToEquiv ![1, 0] ![1, 0])).symm) x

```

lemma `metricTensor_contr_dual_metricTensor_eq_unit`

[\[source\]](#)

The contraction of a metric tensor with its dual via the inner indices gives the unit.

```

lemma metricTensor_contr_dual_metricTensor_eq_unit (c : S.C) :
  {S.metricTensor c | μ v ⊗ S.metricTensor (S.τ c) | v ρ}T.tensor = ({S.unitTensor c | μ ρ}
  |>
  perm (OverColor.equivToHomEq (finMapToEquiv ![1, 0] ![1, 0])
  (metricTensor_contr_dual_metricTensor_perm_cond c))).tensor

```

lemma `metricTensor_contr_dual_metricTensor_outer_eq_unit`

[\[source\]](#)

The contraction of a metric tensor with its dual via the outer indices gives the unit.

```

lemma metricTensor_contr_dual_metricTensor_outer_eq_unit (c : S.C) :
  {S.metricTensor c | v μ ⊗ S.metricTensor (S.τ c) | ρ v}T.tensor = ({S.unitTensor c | μ ρ}
  |>
  perm (OverColor.equivToHomEq
  (finMapToEquiv ![1, 0] ![1, 0]) (fun x => by fin_cases x <;> rfl))).tensor

```

2.28 HepLean.Tensors.TensorSpecies.Pure

[HepLean/Tensors/TensorSpecies/Pure.lean](#) on GitHub.

def Pure [\[source\]](#)

The type of tensors specified by a map to colors $c : \text{OverColor } S.C$.

```
def Pure (c : OverColor S.C) : Type
```

def p [\[source\]](#)

The group action on a pure tensor.

```
def p (g : S.G) (p : Pure S c) : Pure S c
```

def tprod [\[source\]](#)

The underlying tensor of a pure tensor.

```
def tprod (p : Pure S c) : S.F.obj c
```

lemma tprod_equivariant [\[source\]](#)

The map $tprod$ is equivariant with respect to the group action.

```
lemma tprod_equivariant (g : S.G) (p : Pure S c) : (p g p).tprod = (S.F.obj c).p g p.tprod
```

def IsPure [\[source\]](#)

A tensor is pure if it is $\bigotimes [S.k] i, p i$ for some $p : \text{Pure } c$.

```
def IsPure {c : OverColor S.C} (t : S.F.obj c) : Prop
```

lemma zero_isPure [\[source\]](#)

As long as we are dealing with tensors with at least one index, then the zero tensor is pure.

```
lemma zero_isPure {c : OverColor S.C} [h : Nonempty c.left] : @IsPure S c 0
```

lemma tprod_isPure [\[source\]](#)

```
@[simp]
lemma Pure.tprod_isPure {c : OverColor S.C} (p : Pure S c) : S.IsPure p.tprod
```

lemma action_isPure_iff_isPure [\[source\]](#)

```
@[simp]
lemma action_isPure_iff_isPure {c : OverColor S.C} {ψ : S.F.obj c} (g : S.G) :
  S.IsPure ((S.F.obj c).p g ψ) ↔ S.IsPure ψ
```

2.29 HepLean.Tensors.TensorSpecies.UnitTensor

[HepLean/Tensors/TensorSpecies/UnitTensor.lean](#) on GitHub.

def unitTensor[\[source\]](#)

The unit of a tensor species in a PiTensorProduct.

```
def unitTensor (S : TensorSpecies) (c : S.C) : S.F.obj (OverColor.mk ![S.τ c, c])
```

lemma unitTensor_congr[\[source\]](#)

The relation between two units of colors which are equal.

```
lemma unitTensor_congr {c c' : S.C} (h : c = c') : {S.unitTensor c | μ v}^T.tensor =
  (perm (OverColor.equivToHomEq (Equiv.refl _) (fun x => by subst h; fin_cases x <;> rfl))
    {S.unitTensor c' | μ v}^T).tensor
```

lemma pairIsoSep_inv_unitTensor[\[source\]](#)

Applying Discrete.pairIsoSep inv to unitTensor returns the unit natural transformation evaluated at 1.

```
lemma pairIsoSep_inv_unitTensor (c : S.C) :
  (Discrete.pairIsoSep S.FD).inv.hom (S.unitTensor c) =
  (S.unit.app (Discrete.mk c)).hom (1 : S.k)
```

lemma unitTensor_eq_dual_perm[\[source\]](#)

The unit tensor is equal to a permutation of indices of the dual tensor.

```
lemma unitTensor_eq_dual_perm (c : S.C) : {S.unitTensor c | μ v}^T.tensor =
  ({S.unitTensor (S.τ c) | v μ}^T |>
    perm (OverColor.equivToHomEq (finMapToEquiv ![1,0] ![1, 0])
      (fun x => match x with | 0 => by rfl | 1 => (S.τ_involution c).symm))).tensor
```

lemma dual_unitTensor_eq_perm[\[source\]](#)

The unit tensor of the dual of a color c is equal to the unit tensor of c with indices permuted.

```
lemma dual_unitTensor_eq_perm (c : S.C) :
  {S.unitTensor (S.τ c) | v μ}^T.tensor = ({S.unitTensor c | μ v}^T |>
    perm (OverColor.equivToHomEq (finMapToEquiv ![1, 0] ![1, 0])
      (fun x => match x with | 0 => (S.τ_involution c) | 1 => by rfl))).tensor
```

lemma contrOneTwoLeft_unitTensor[\[source\]](#)

Applying contrOneTwoLeft with the unit tensor is the identity map.

```
lemma contrOneTwoLeft_unitTensor {c1 : S.C} (x : S.F.obj (OverColor.mk ![c1])) :
  contrOneTwoLeft x (S.unitTensor c1) = x
```

lemma vec_contr_unitTensor_eq_self[\[source\]](#)

Contracting a vector with a unit tensor returns the vector.

```
lemma vec_contr_unitTensor_eq_self {c1 : S.C} (x : S.F.obj (OverColor.mk ![c1])) :
  {x | μ ⊗ S.unitTensor c1 | μ v}^T.tensor = ({x | v}^T |>
    perm (OverColor.equivToHomEq (Equiv.refl _) (fun x => by fin_cases x; rfl))).tensor
```

lemma unitTensor_contr_vec_eq_self[\[source\]](#)

Contracting a unit tensor with a vector returns the unit vector.

```
lemma unitTensor_contr_vec_eq_self {c1 : S.C} (x : S.F.obj (OverColor.mk ![S.τ c1])) :
  {S.unitTensor c1 | v μ ⊗ x | μ}^T.tensor = ({x | v}^T |>
    perm (OverColor.equivToHomEq (Equiv.refl _) (fun x => by fin_cases x; rfl))).tensor
```

2.30 HepLean.Tensors.Tree.NodeIdentities.Assoc

[HepLean/Tensors/Tree/NodeIdentities/Assoc.lean](#) on GitHub.

lemma assoc_one_two_two[\[source\]](#)

The associativity lemma for $t1 \mid \mu \otimes t2 \mid \mu \nu \otimes t3 \mid \nu \sigma$.

```
lemma assoc_one_two_two {c1 c2 c3 : S.C} (t1 : S.F.obj (OverColor.mk ![c1]))
  (t2 : S.F.obj (OverColor.mk ![S.τ c1, c2])) (t3 : S.F.obj (OverColor.mk ![S.τ c2, c3])) :
  {t1 | μ ⊗ t2 | μ ν ⊗ t3 | ν σ}^T.tensor = ({t1 | μ ⊗ (t2 | μ ν ⊗ t3 | ν σ)}^T
    |> perm (OverColor.equivToHomEq (Equiv.refl (Fin 1)) (by
      intro x
      fin_cases x
      rfl))).tensor
```

2.31 HepLean.Tensors.Tree.NodeIdentities.Basic

[HepLean/Tensors/Tree/NodeIdentities/Basic.lean](#) on GitHub.

lemma tensorNode_of_tree[\[source\]](#)

The tenor node of a tensor tree's tensor has a tensor which is the tensor tree's tensor.

```
lemma tensorNode_of_tree (t : TensorTree S c) : (tensorNode t.tensor).tensor = t.tensor
```

2.32 HepLean.Tensors.Tree.NodeIdentities.ContrContr

[HepLean/Tensors/Tree/NodeIdentities/ContrContr.lean](#) on GitHub.

lemma contrContrPerm_hom_left_apply[\[source\]](#)

```
@[simp]
lemma contrContrPerm_hom_left_apply {n : ℕ} {c : Fin n.succ.succ.succ.succ → S.C}
  {i : Fin n.succ.succ.succ.succ}
  {j : Fin n.succ.succ.succ} {k : Fin n.succ.succ} {l : Fin n.succ}
  (hij : c (i.succAbove j) = S.τ (c i)) (hkl : (c ◦ i.succAbove ◦ j.succAbove) (k.succAbove l)
    ) =
  S.τ ((c ◦ i.succAbove ◦ j.succAbove) k)) (x : Fin n) :
  (contrContrPerm hij hkl).hom.left x = x
```

2.33 HepLean.Tensors.Tree.NodeIdentities.ContrSwap

[HepLean/Tensors/Tree/NodeIdentities/ContrSwap.lean](#) on GitHub.

lemma contrSwapHom_toEquiv[\[source\]](#)

@[simp]

lemma contrSwapHom_toEquiv : Hom.toEquiv q.contrSwapHom = Equiv.refl (Fin n)**lemma contrSwapHom_hom_left_apply**[\[source\]](#)

@[simp]

lemma contrSwapHom_hom_left_apply (x : Fin n) : q.contrSwapHom.hom.left x = x**2.34 HepLean.Tensors.Tree.NodeIdentities.PermProd**[HepLean/Tensors/Tree/NodeIdentities/PermProd.lean](#) on GitHub.**lemma permProdLeft_toEquiv**[\[source\]](#)

@[simp]

lemma permProdLeft_toEquiv : Hom.toEquiv (permProdLeft c2 σ) = finSumFinEquiv.symm.trans
(((Hom.toEquiv σ).sumCongr (Equiv.refl (Fin n2))).trans finSumFinEquiv)**lemma permProdRight_toEquiv**[\[source\]](#)

@[simp]

lemma permProdRight_toEquiv : Hom.toEquiv (permProdRight c2 σ) = finSumFinEquiv.symm.trans
(((Equiv.refl (Fin n2)).sumCongr (Hom.toEquiv σ)).trans finSumFinEquiv)**2.35 HepLean.Tensors.Tree.NodeIdentities.ProdAssoc**[HepLean/Tensors/Tree/NodeIdentities/ProdAssoc.lean](#) on GitHub.**lemma assocPerm_toHomEquiv_hom**[\[source\]](#)

@[simp]

lemma assocPerm_toHomEquiv_hom : Hom.toEquiv (assocPerm c c2 c3).hom = finSumFinEquiv.symm.
trans
(((finSumFinEquiv.symm.sumCongr (Equiv.refl (Fin n3))).trans
((Equiv.sumAssoc (Fin n) (Fin n2) (Fin n3)).trans
((Equiv.refl (Fin n)).sumCongr finSumFinEquiv).trans finSumFinEquiv)))**lemma assocPerm_toHomEquiv_inv**[\[source\]](#)

@[simp]

lemma assocPerm_toHomEquiv_inv : Hom.toEquiv (assocPerm c c2 c3).inv = finSumFinEquiv.symm.
trans
(((Equiv.refl (Fin n)).sumCongr finSumFinEquiv.symm).trans
((Equiv.sumAssoc (Fin n) (Fin n2) (Fin n3)).symm.trans
((finSumFinEquiv.sumCongr (Equiv.refl (Fin n3))).trans finSumFinEquiv)))**2.36 HepLean.Tensors.Tree.NodeIdentities.ProdComm**[HepLean/Tensors/Tree/NodeIdentities/ProdComm.lean](#) on GitHub.

lemma braidPerm_toEquiv[\[source\]](#)

```
@[simp]
lemma braidPerm_toEquiv : Hom.toEquiv (braidPerm c c2) = finSumFinEquiv.symm.trans
  ((Equiv.sumComm (Fin n2) (Fin n)).trans finSumFinEquiv)
```

2.37 HepLean.Tensors.Tree.NodeIdentities.ProdContr

[HepLean/Tensors/Tree/NodeIdentities/ProdContr.lean](#) on GitHub.

lemma contrMap_prod_tprod_aux[\[source\]](#)

```
lemma contrMap_prod_tprod_aux
  (l : (OverColor.mk (c ◦ q.i.succAbove ◦ q.j.succAbove)).left @ (OverColor.mk c1).left)
  (l' : Fin n.succ.succ @ Fin n1)
  (h : Sum.elim c c1 l' = Sum.elim (c ◦ q.i.succAbove ◦ q.j.succAbove) c1 l)
  (h' : l' = (Sum.map (q.i.succAbove ◦ q.j.succAbove) id l))
  (p : (i : (1 Type).obj (OverColor.mk c).left) →
    CoeSort.coe (S.FD.obj { as := (OverColor.mk c).hom i }))
  (q' : (i : (1 Type).obj (OverColor.mk c1).left) →
    CoeSort.coe (S.FD.obj { as := (OverColor.mk c1).hom i })) :
  (lift.discreteSumEquiv S.FD l)
  (HepLean.PiTensorProduct.elimPureTensor
  (fun k => p (q.i.succAbove (q.j.succAbove k))) q' l) =
  (S.FD.map (eqToHom (by simp [h]))).hom
  ((lift.discreteSumEquiv S.FD l'))
  (HepLean.PiTensorProduct.elimPureTensor p q' l'))
```

2.38 scripts.MetaPrograms.no_docs

[scripts/MetaPrograms/no_docs.lean](#) on GitHub.

def NoDocStringDef[\[source\]](#)

```
def Imports.NoDocStringDef (imp : Import) : MetaM UInt32
```

def NoDocStringLemma[\[source\]](#)

```
def Imports.NoDocStringLemma (imp : Import) : MetaM UInt32
```

2.39 scripts.stats

[scripts/stats.lean](#) on GitHub.

def toHtml[\[source\]](#)

```
unsafe def Stats.toHtml : MetaM String
```

2.40 scripts.Template

[scripts/Template.lean](#) on GitHub.

def baseHtmlGenerator[\[source\]](#)

The HTML template used for all pages.

```
def baseHtmlGenerator (title : String) (site : Array Html) : BaseHtmlM Html
```

def baseHtml[\[source\]](#)

A comfortability wrapper around baseHtmlGenerator.

```
def baseHtml (title : String) (site : Html) : BaseHtmlM Html
```